ASSIGNMENT 3 - TASK 1

Multi-Class Classification Using Fully Connected Neural Networks

Student Name	Sushmita Halder, Jyoti, Rajeev Kumar
Roll Number	2621MA08, 2621MA12, 2621MA01
Group	08
Course	Large Language Models
Submission Date	15/09/2026

1. Introduction

This task investigates multi-class classification using Fully Connected Neural Networks (FCNNs) implemented from scratch. Two two-dimensional datasets were used: a three-class Gaussian dataset and a three-class spiral dataset. The purpose of the experiments was to study how neural-network architecture affects classification performance and to compare the behavior of a simple single-neuron classifier with a multi-layer FCNN.

2. Objectives

- Generate and classify a three-class Gaussian dataset.
- Generate and classify a three-class nonlinear spiral dataset.
- Split the datasets into training, validation, and testing subsets.
- Implement forward propagation, backpropagation, and SGD from scratch.
- Compare different FCNN architectures.
- Select the best architecture using validation accuracy.
- Evaluate the selected architecture using independent test data.
- Visualize error curves, decision regions, hidden-layer outputs, and output-node behavior.
- Compare the FCNN with a single-neuron model.

3. Experiment 1: Gaussian Dataset

3.1 Dataset Generation

The Gaussian experiment used three classes with 500 samples per class, giving a total of 1500 examples. The final notebook output confirmed an input matrix of shape (1500, 2) and a label vector of shape (1500,).

Output	Value
Number of classes	3
Samples per class	500
Total samples	1500
Input shape	(1500, 2)
Label shape	(1500,)

Visualization of the generated three-class Gaussian dataset.



3.2 Data Splitting and Preprocessing

The data was divided into training, validation, and test sets. The notebook output produced 900 training samples, 300 validation samples, and 300 test samples, corresponding to the required 60%, 20%, and 20% split.

Dataset Portion	Number of Samples	Percentage
Training	900	60%
Validation	300	20%
Testing	300	20%

One-hot encoding was used for the target labels so that the three output neurons could represent the three classes.

3.3 Gaussian FCNN Architecture Experiment

A one-hidden-layer FCNN was evaluated using four different hidden-layer sizes: 3, 5, 8, and 10 hidden nodes. The architecture notation 2-H-3 represents two input nodes, H hidden nodes, and three output nodes.

Architecture	Hidden Nodes	Validation Accuracy	Final Average Error
2-3-3	3	100.00%	0.005642
2-5-3	5	100.00%	0.003676
2-8-3	8	100.00%	0.003732
2-10-3	10	100.00%	0.003485

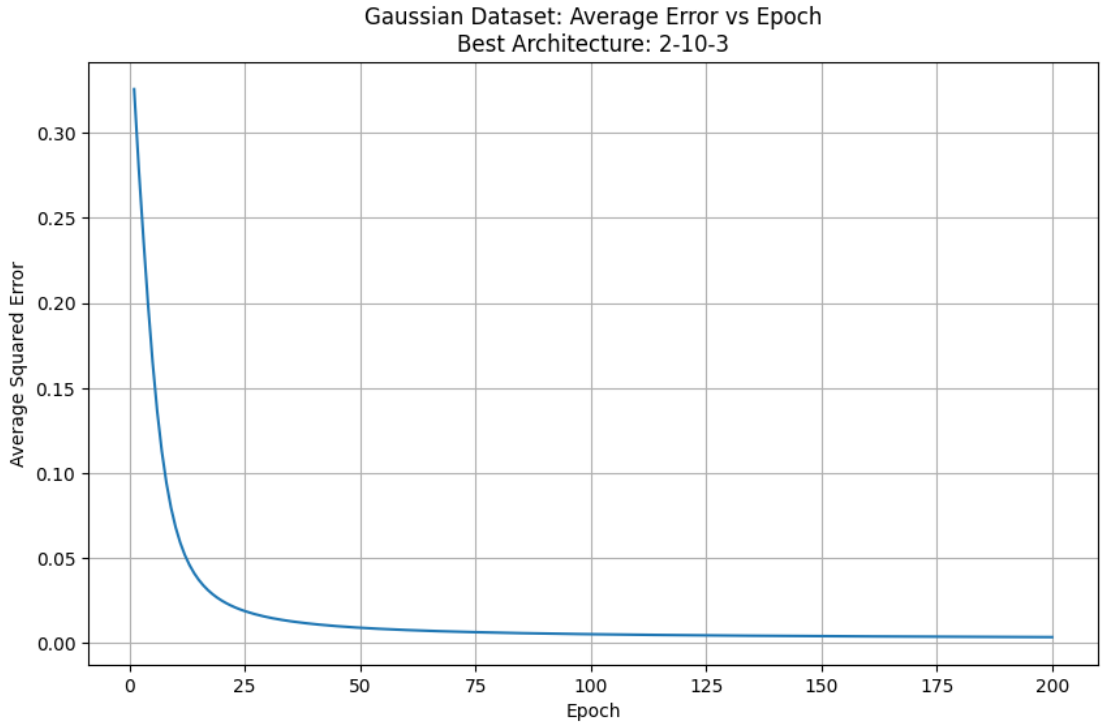
3.4 Best Gaussian Architecture

All four architectures achieved the same validation accuracy of 100%. Therefore, validation accuracy alone produced a tie. The final average error was used as a secondary criterion. The 2-10-3 architecture had the lowest final average error and was selected as the best Gaussian FCNN.

Selected Criterion	Result
Best number of hidden nodes	10
Validation accuracy	100.00%
Final average error	0.0034846044

3.5 Average Error Versus Epochs

The following plot shows the learning behavior of the selected Gaussian architecture. The decreasing average error indicates successful learning during training.



Forward and Backward Propagation

The neural network training process consists of two main steps: Forward Propagation and Backpropagation.

1. Forward Propagation

In forward propagation, the input data passes through the hidden layer and then the output layer to generate a prediction.

Hidden layer calculation:

Hidden Input = $W_1X + b_1$

Hidden Output = Sigmoid(Hidden Input)

Output layer calculation:

Output Input = $W_2 \times$Hidden Output + b_2

Predicted Output = Sigmoid(Output Input)

The predicted output is then compared with the target value.

2. Backpropagation

Backpropagation calculates the error in the prediction and propagates the error backward through the network to update the weights and biases.

Output Error:

Output Error = Predicted Output − Target

Output Delta:

Output Delta = Output Error × Sigmoid Derivative(Output)

For the sigmoid activation function:

Sigmoid Derivative = Output × (1 − Output)

Gradient of output layer weights:

dW_2 = Hidden Output × Output Delta

Gradient of output layer bias:

db_2 = Output Delta

Hidden layer error:

Hidden Error = W_2 × Output Delta

Hidden layer delta:

Hidden Delta = Hidden Error × Sigmoid Derivative(Hidden Output)

Gradient of hidden layer weights:

dW_1 = Input × Hidden Delta

Gradient of hidden layer bias:

db_1 = Hidden Delta

3. Weight and Bias Update

The weights and biases are updated using Gradient Descent:

$W_2 = W_2$ − Learning Rate × dW_2

$b_2 = b_2$ − Learning Rate × db_2

$W_1 = W_1$ − Learning Rate × dW_1

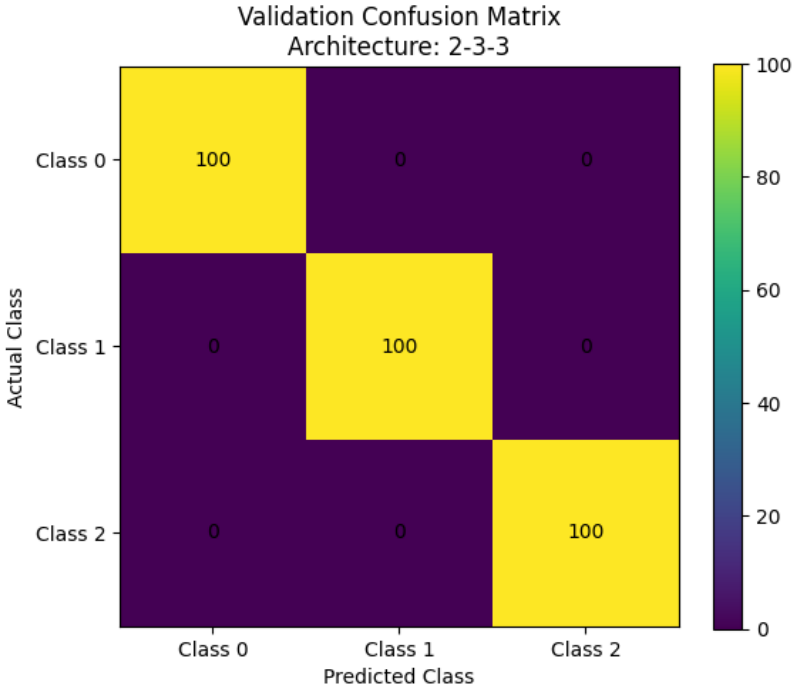
$b_1 = b_1$ − Learning Rate × db_1

Backpropagation repeats this process for multiple iterations. In each iteration, the network calculates the prediction error, propagates the error backward, calculates the gradients, and updates the weights and biases to minimize the prediction error.

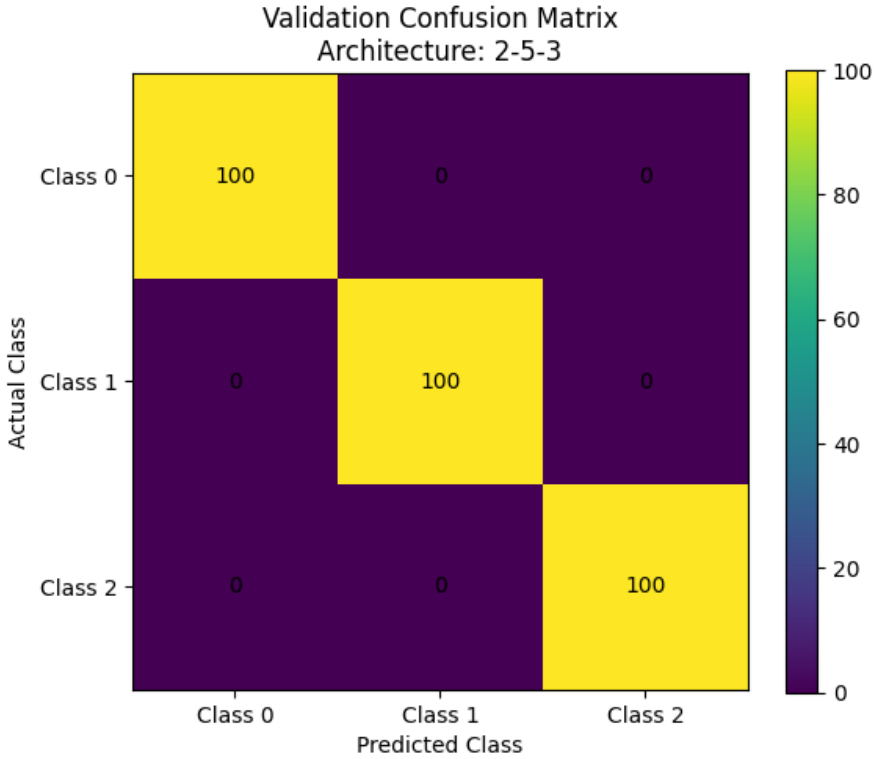
3.6 Validation Performance and Confusion Matrices

Each Gaussian architecture was evaluated on the validation set. All architectures correctly classified all 300 validation examples, producing 100% validation accuracy.

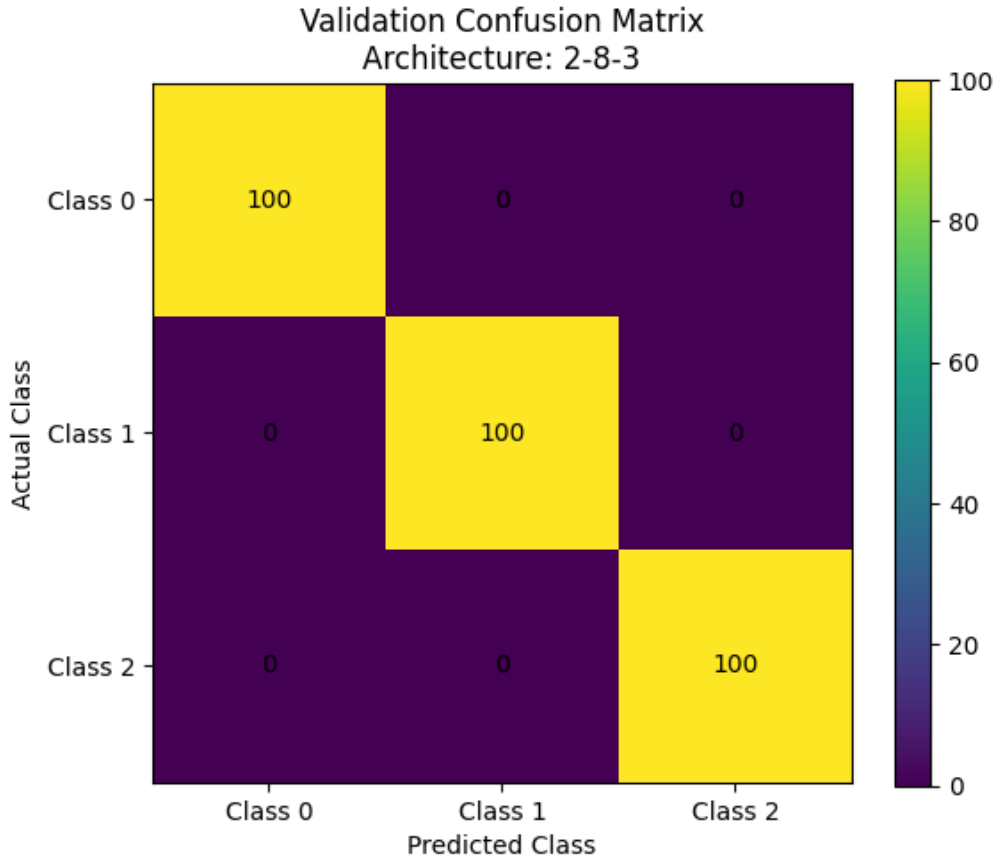
Validation confusion matrix for Gaussian architecture 2-3-3.



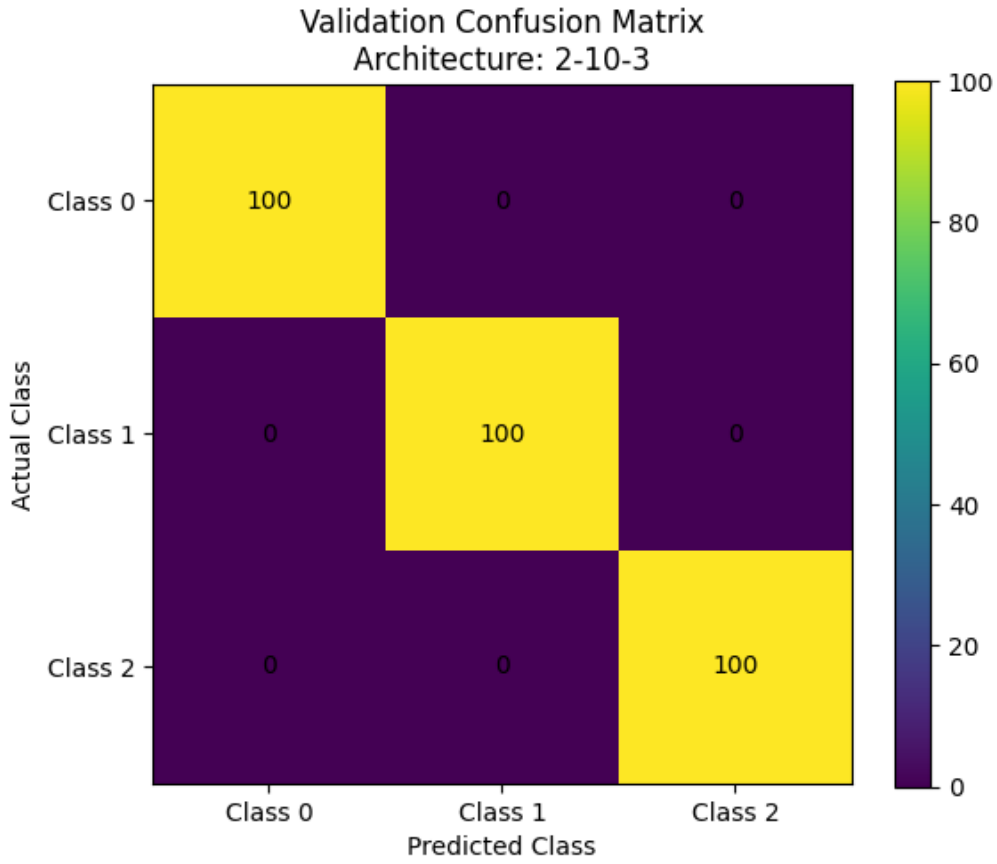
Validation confusion matrix for Gaussian architecture 2-5-3.



Validation confusion matrix for Gaussian architecture 2-8-3.



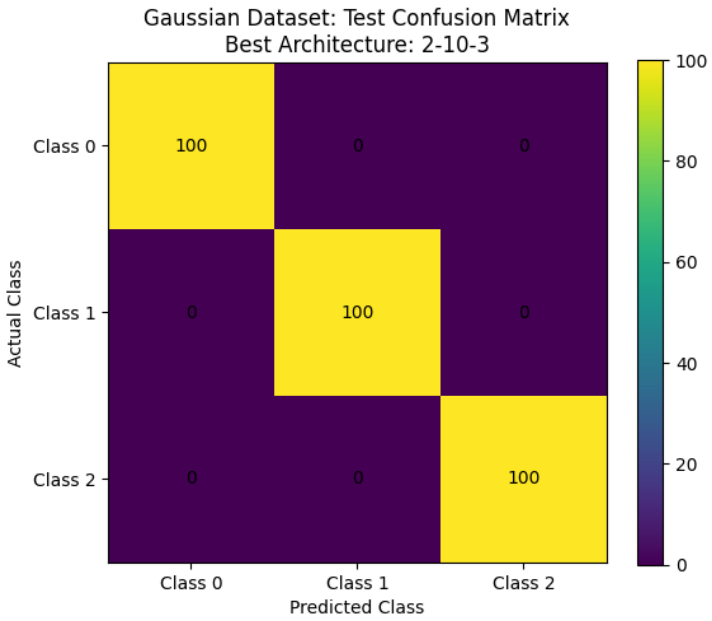
Validation confusion matrix for Gaussian architecture 2-10-3.



3.7 Final Gaussian Test Results

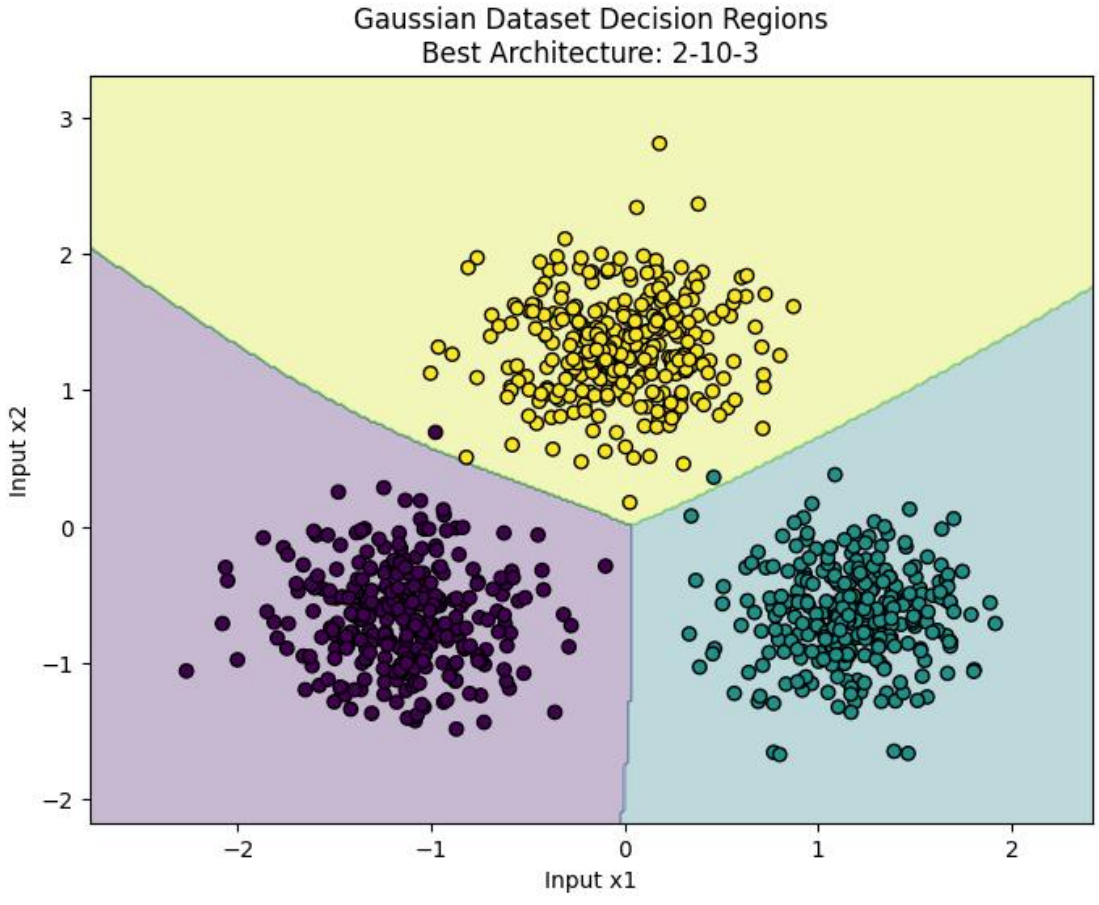
Measure	Result
Best architecture	2-10-3
Test accuracy	100.00%
Test samples	300

The test confusion matrix was:



3.8 Gaussian Decision Region

The decision-region plot illustrates how the selected FCNN divides the two-dimensional input space into three class regions



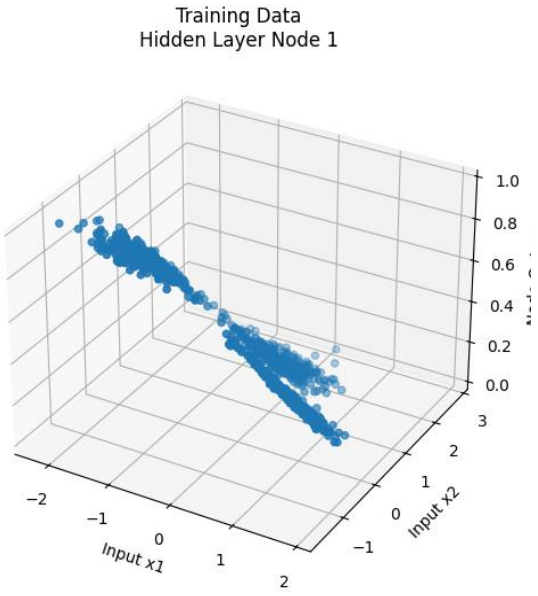
3.9 Gaussian Hidden and Output Node Visualizations

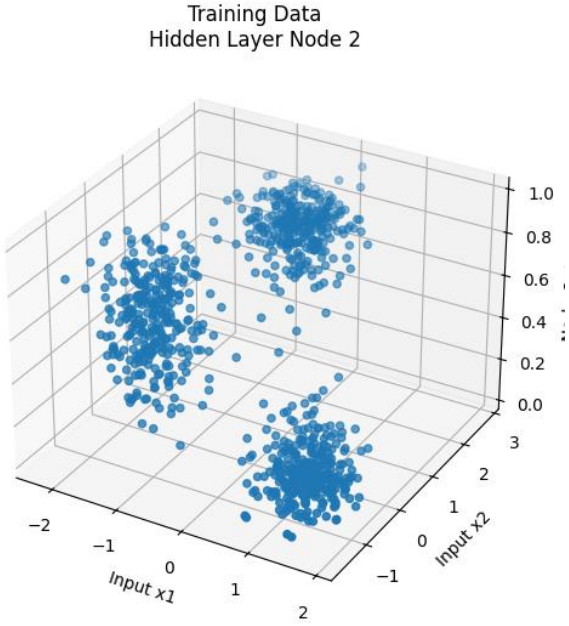
Generated visualizations for the hidden and output nodes over the training, validation, and test data. All generated figures are included below as recorded outputs.

Gaussian Training Data Node Visualizations

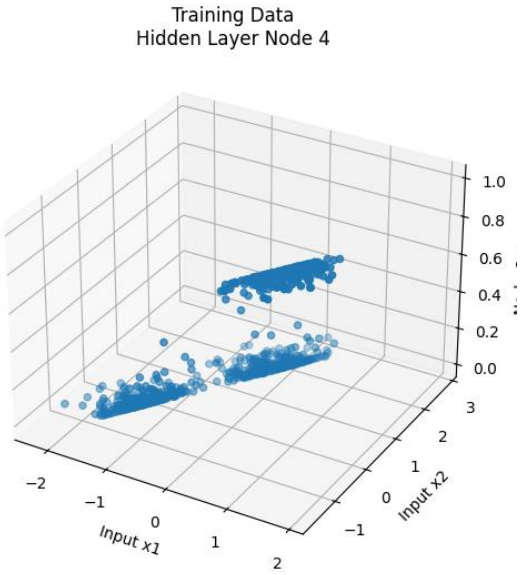
Gaussian Training Data Node Visualizations

Gaussian Training Data Node Visualizations –

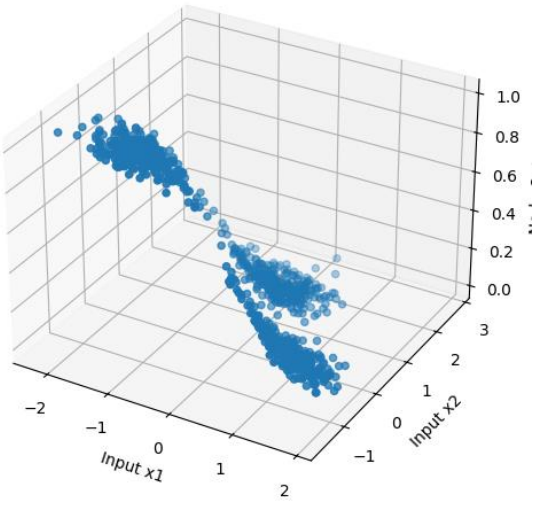




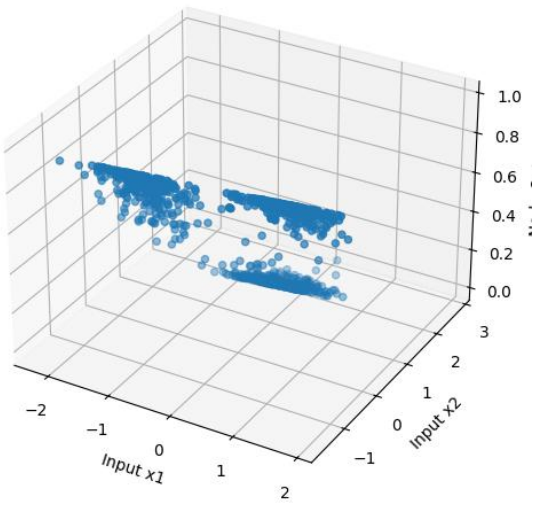




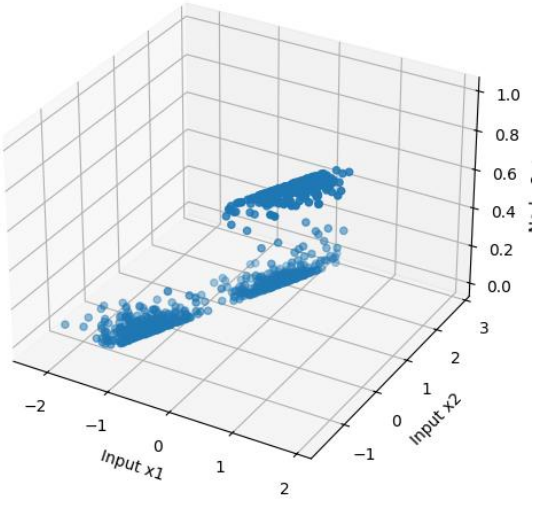
Training Data
Hidden Layer Node 5



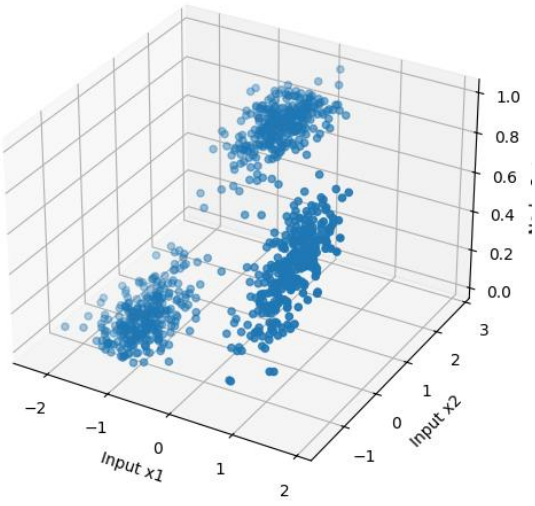
Training Data
Hidden Layer Node 6



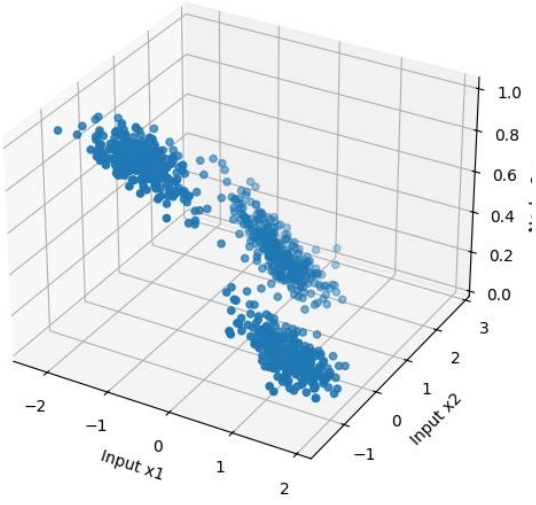
Training Data
Hidden Layer Node 7



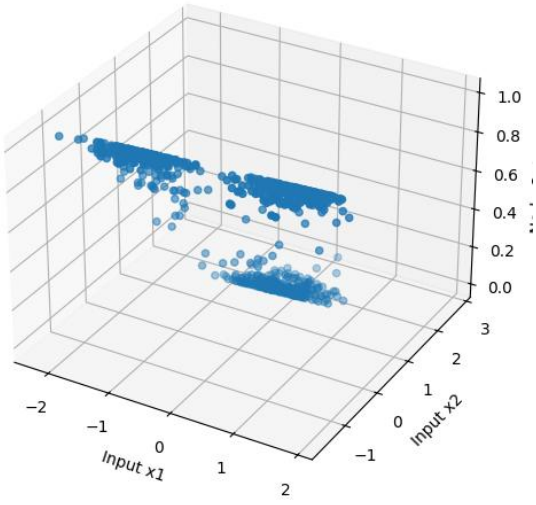
Training Data
Hidden Layer Node 8

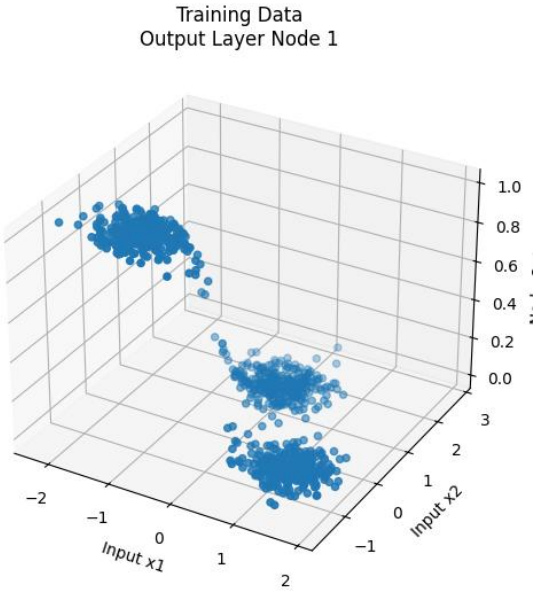


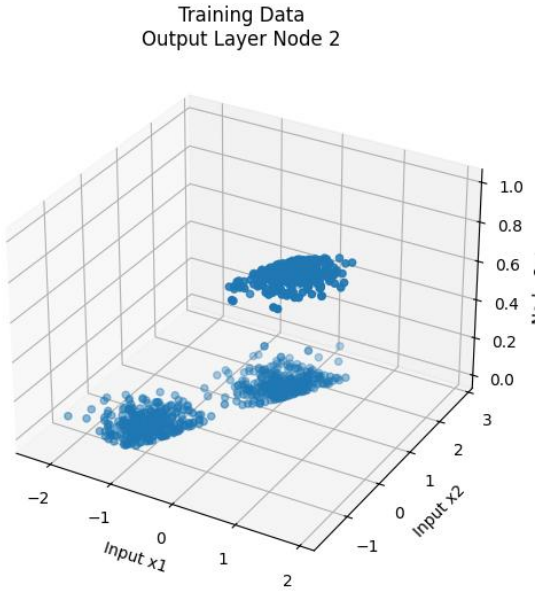
Training Data
Hidden Layer Node 9

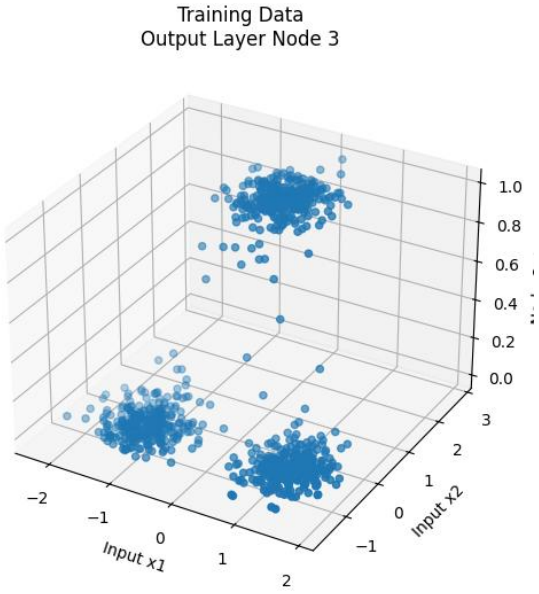


Training Data
Hidden Layer Node 10

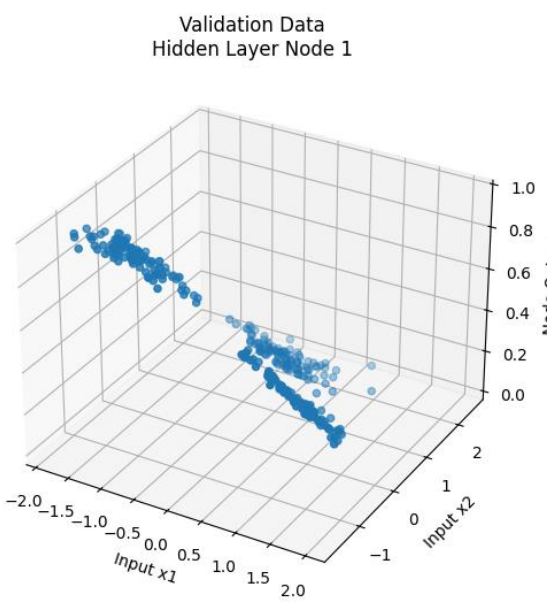


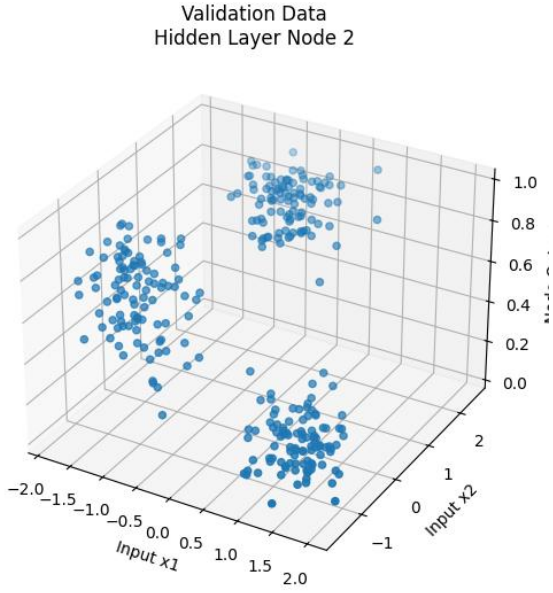


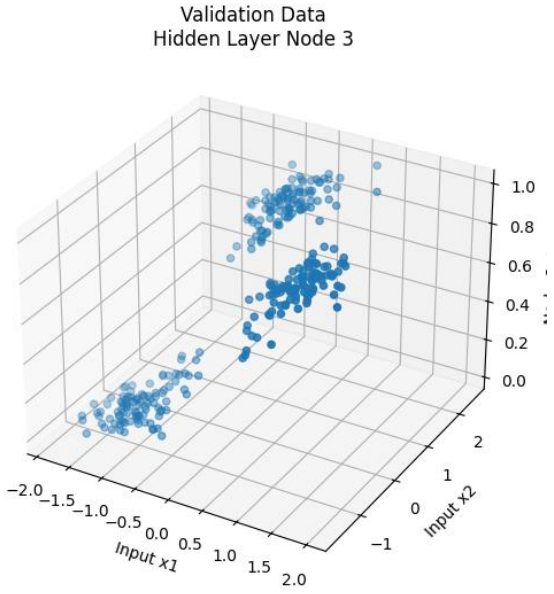


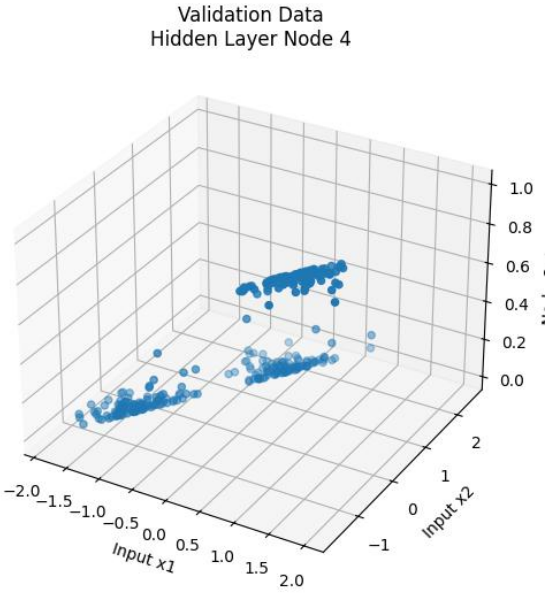


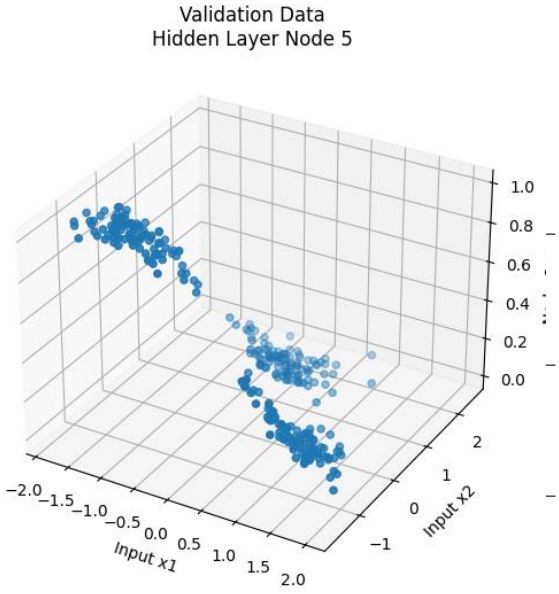
Gaussian Validation Data Node Visualizations



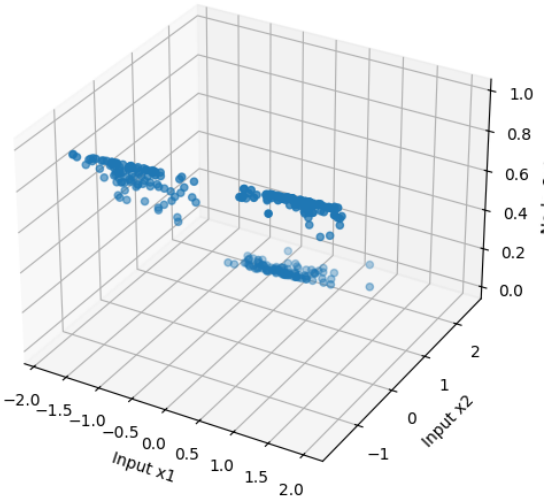




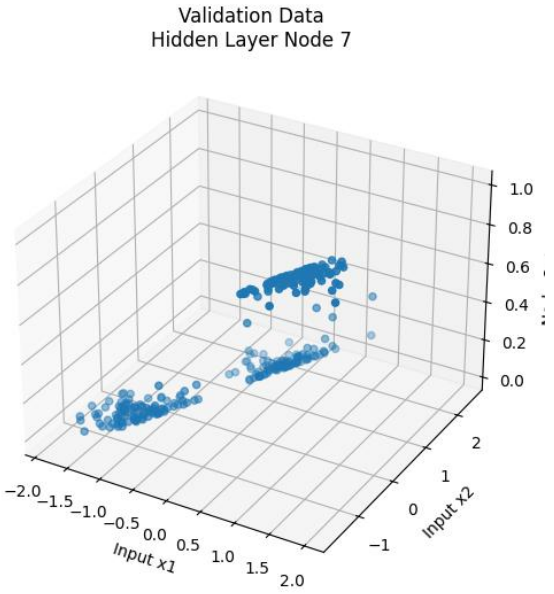




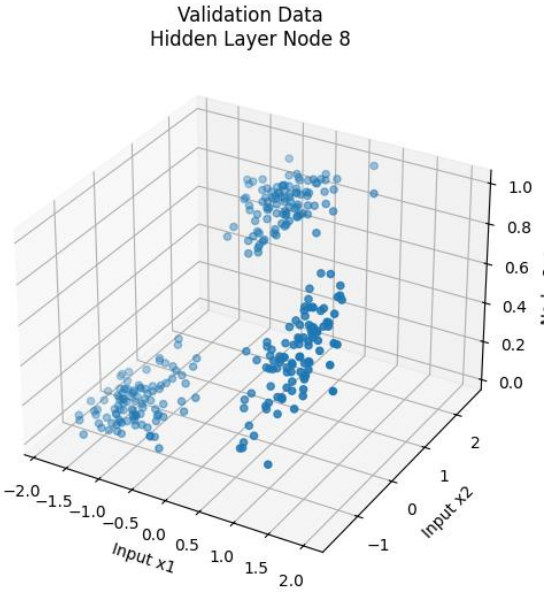
- Figure 8



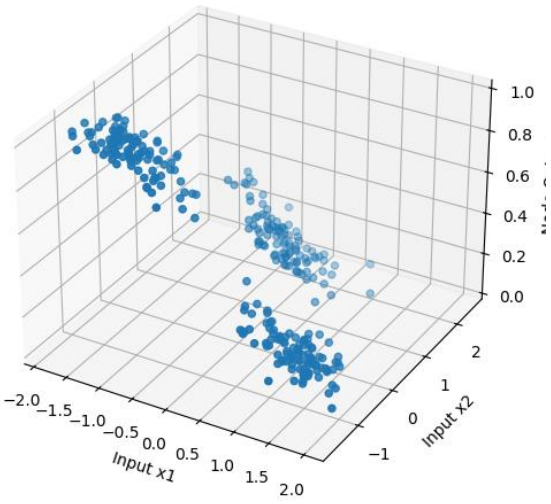
- Figure 9



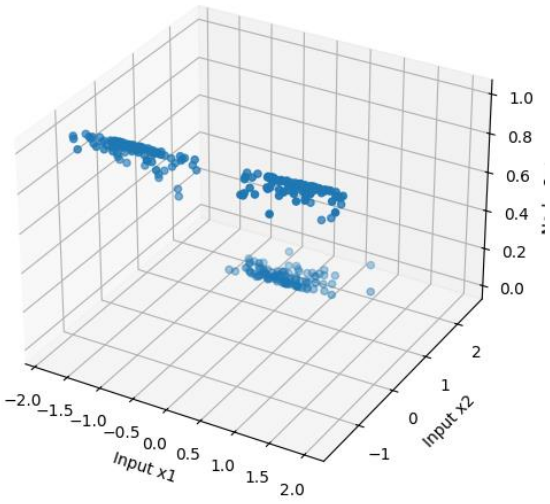
- Figure 10



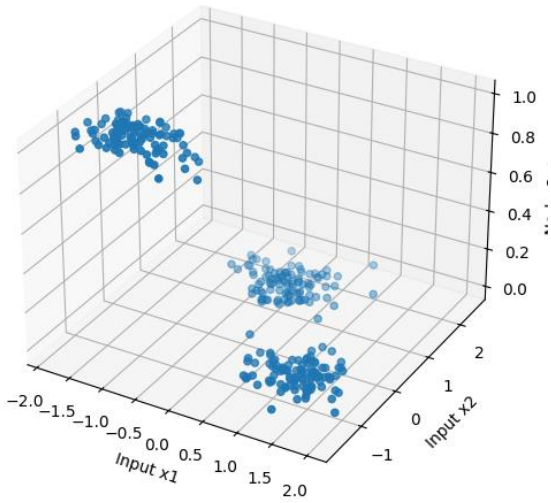
Validation Data
Hidden Layer Node 9



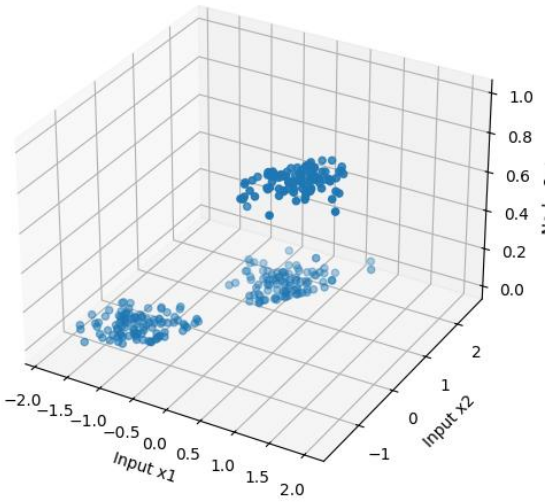
Validation Data
Hidden Layer Node 10



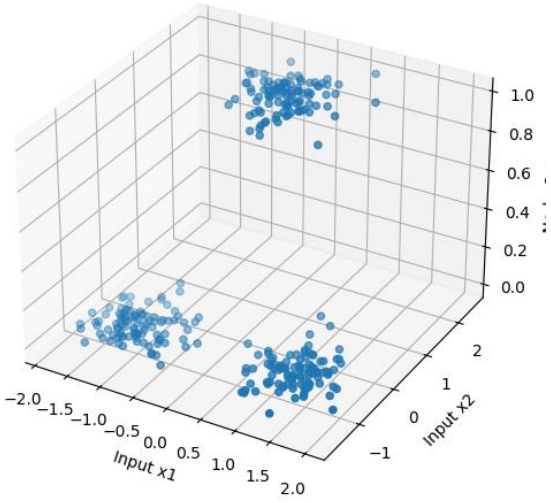
Validation Data
Output Layer Node 1



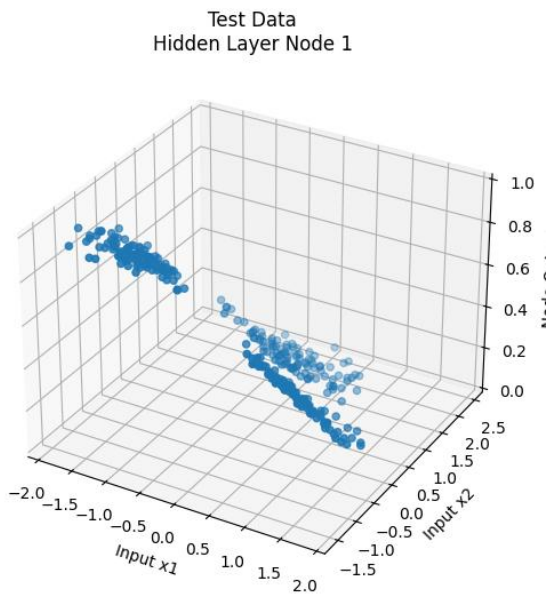
Validation Data
Output Layer Node 2

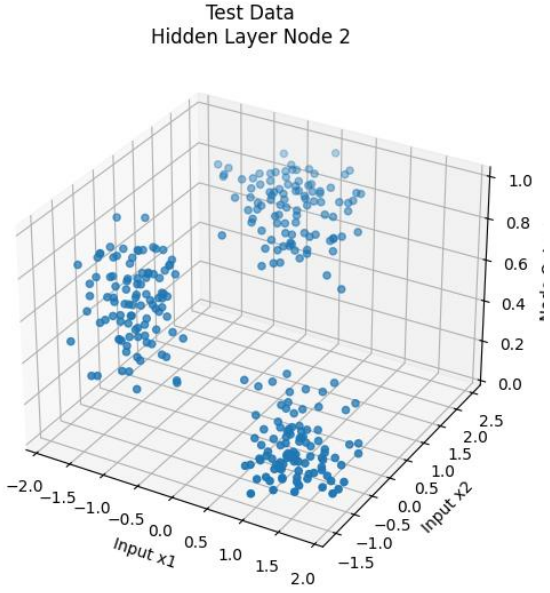


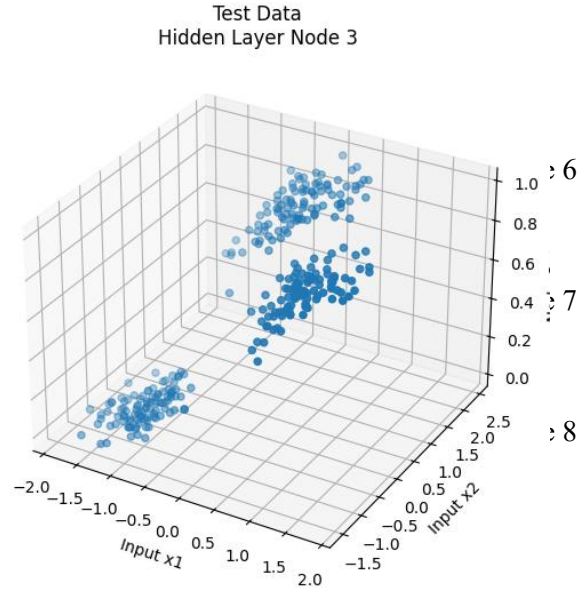
Validation Data
Output Layer Node 3

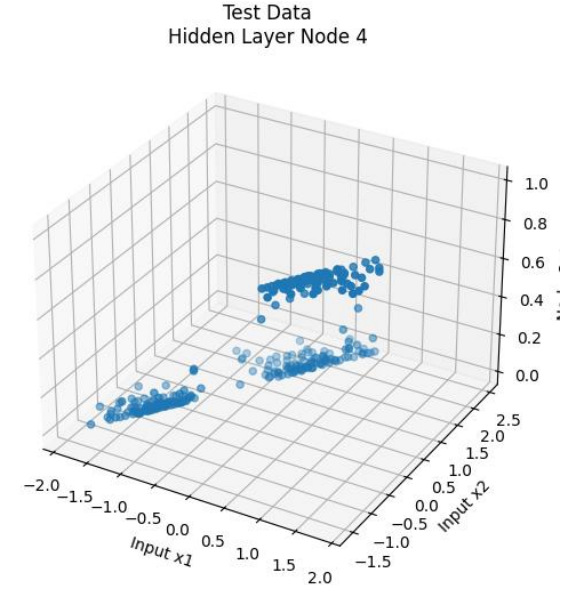


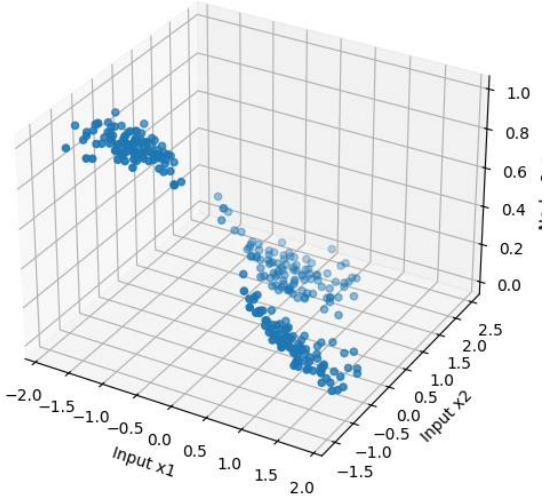
Gaussian Test Data Node Visualizations

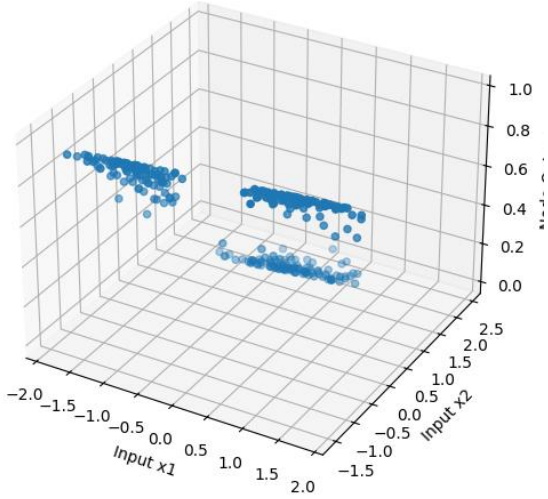




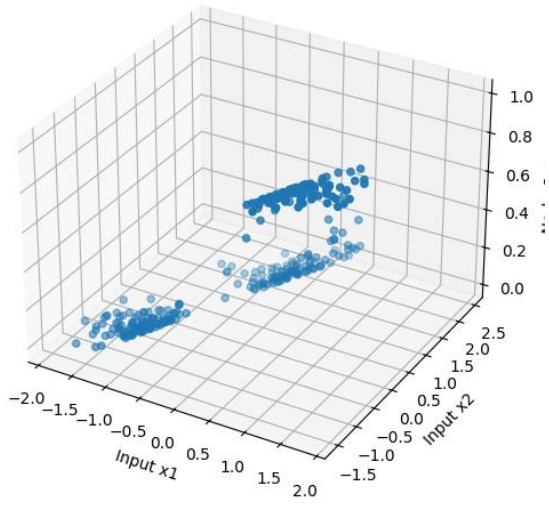




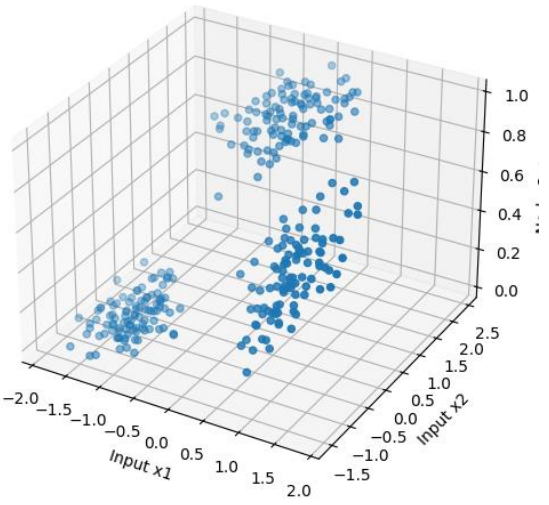




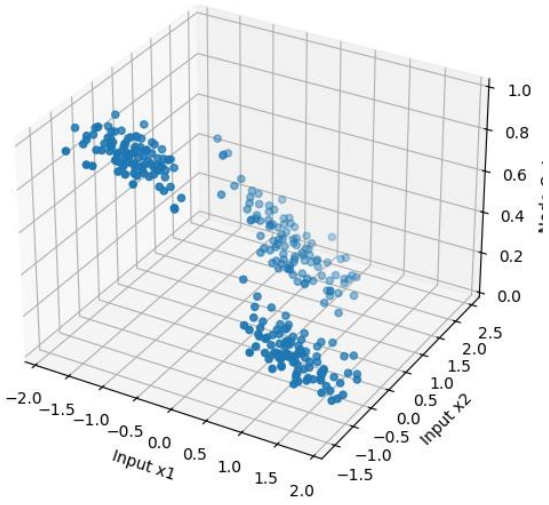
Test Data
Hidden Layer Node 7



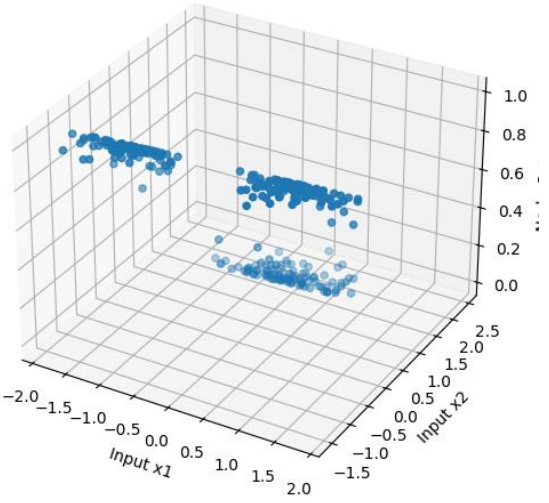
Test Data
Hidden Layer Node 8



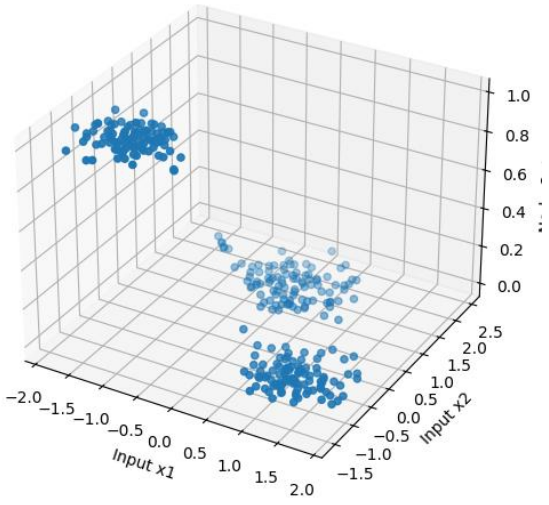
Test Data
Hidden Layer Node 9



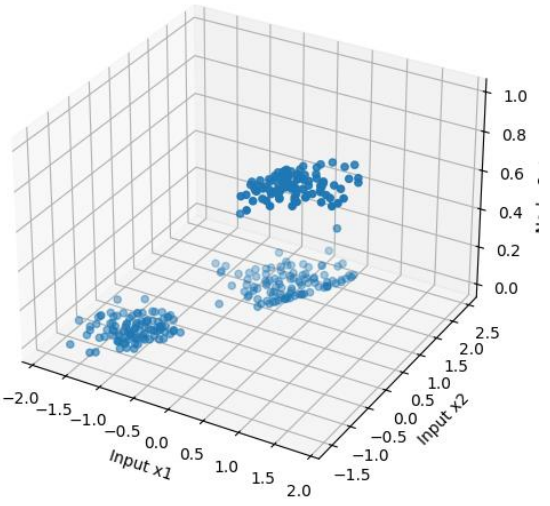
Test Data
Hidden Layer Node 10



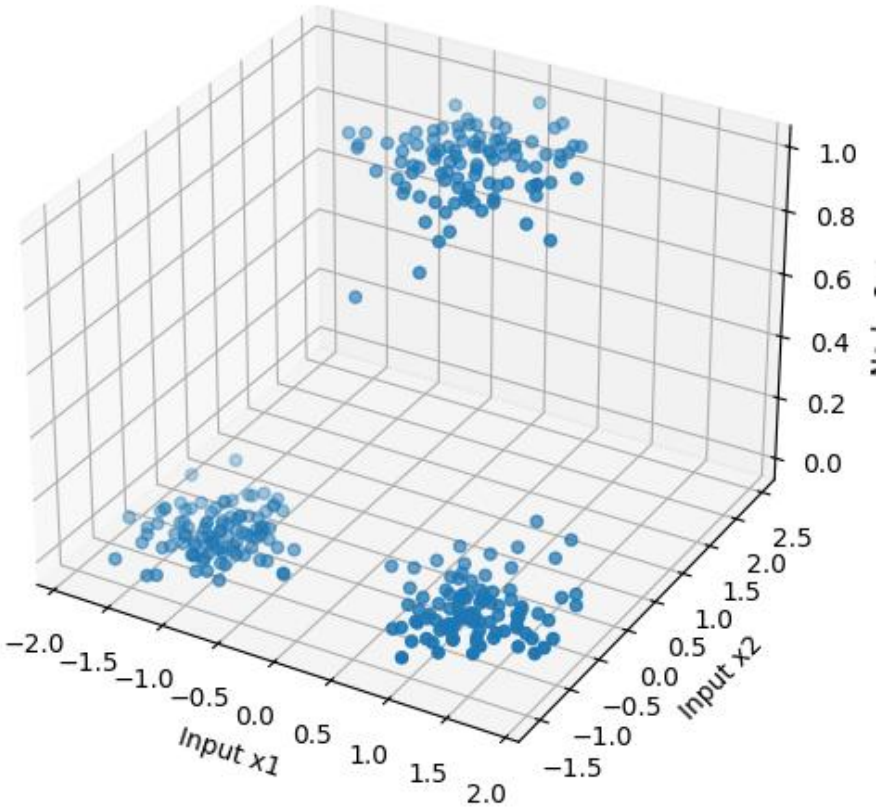
Test Data
Output Layer Node 1



Test Data
Output Layer Node 2



Test Data
Output Layer Node 3



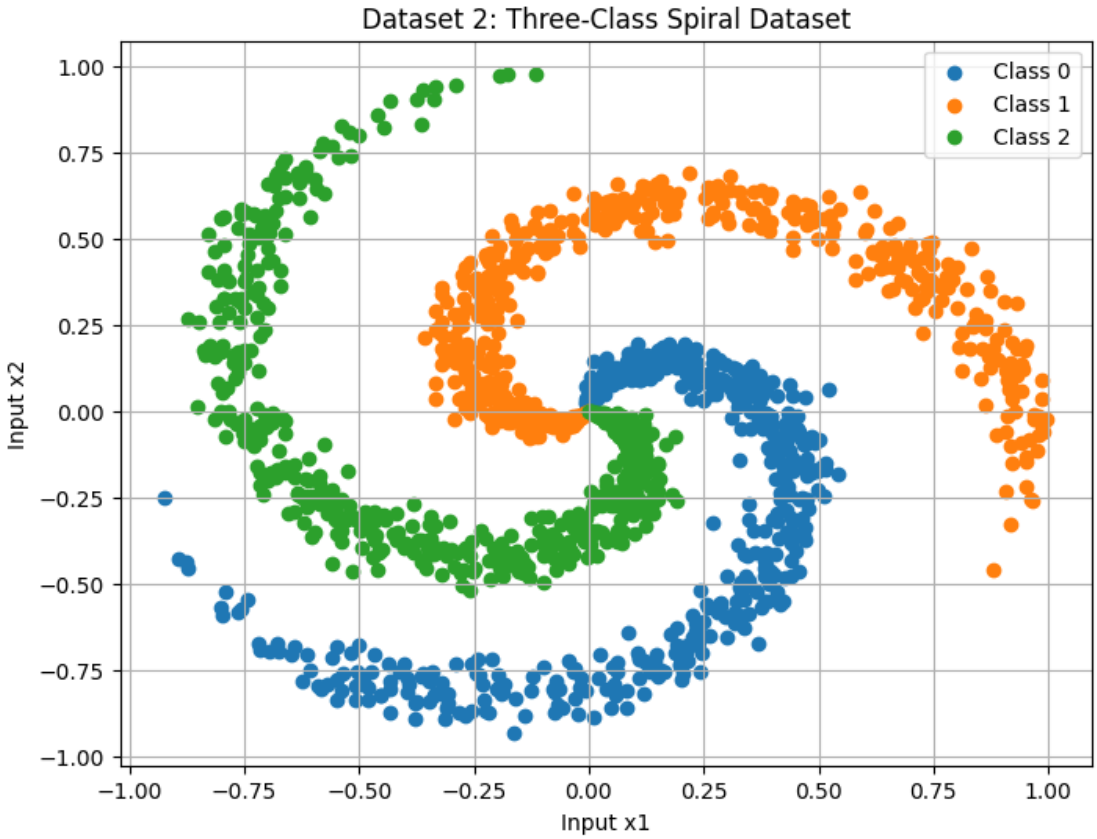
4. Experiment 2: Spiral Dataset

4.1 Dataset Description

The spiral dataset also contained 1500 examples arranged into three classes. Unlike the Gaussian dataset, the class regions are nonlinear and interleaving, making the classification problem substantially more difficult.

Output	Value
Input shape	(1500, 2)
Label shape	(1500,)
Number of classes	3

Figure. Visualization of the three-class spiral dataset.



4.2 Spiral FCNN Architecture Experiment

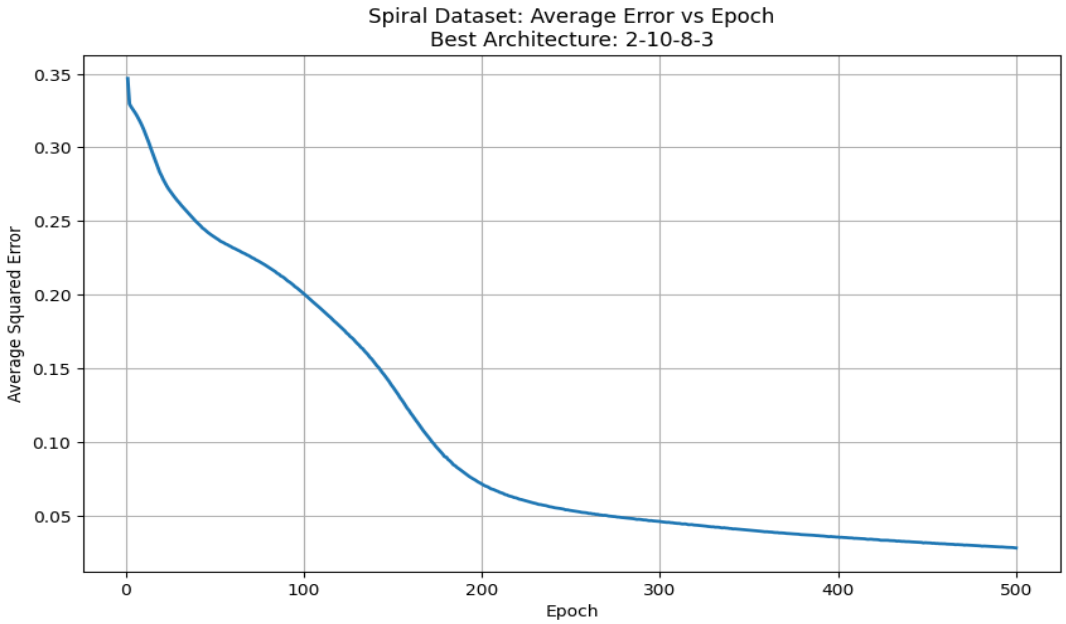
A deeper FCNN with two hidden layers was used for the nonlinear spiral problem. Four architectures were evaluated.

Architecture	Hidden Layer 1	Hidden Layer 2	Validation Accuracy	Final Average Error
2-5-3-3	5	3	96.67%	0.045307
2-8-5-3	8	5	96.67%	0.028155
2-10-5-3	10	5	96.33%	0.032461
2-10-8-3	10	8	97.00%	0.027944

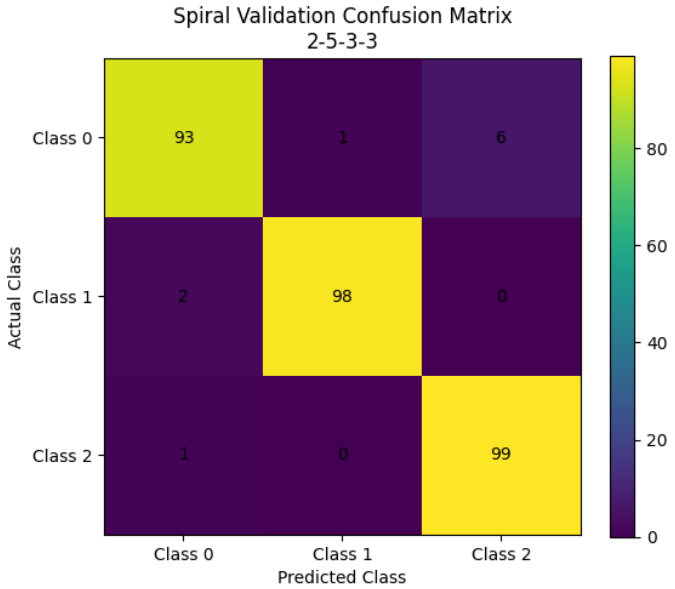
4.3 Best Spiral Architecture

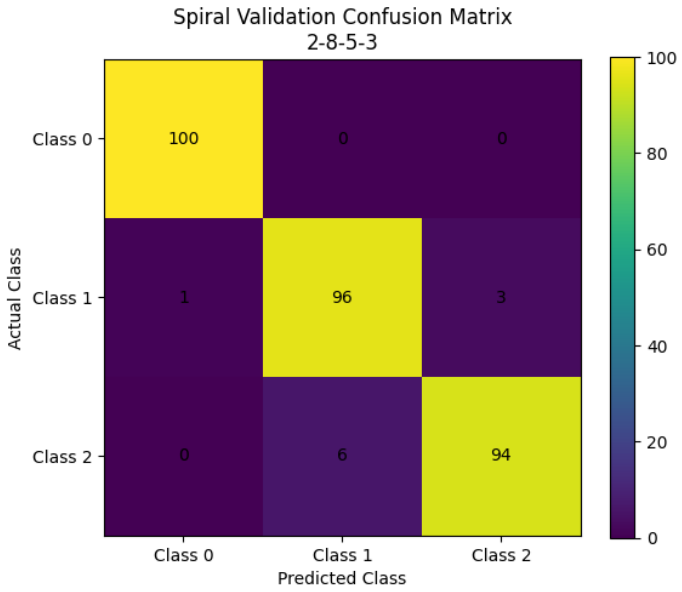
The architecture 2-10-8-3 achieved the highest validation accuracy of 97.00% and also had the lowest final average error among the tested configurations. It was therefore selected as the best architecture.

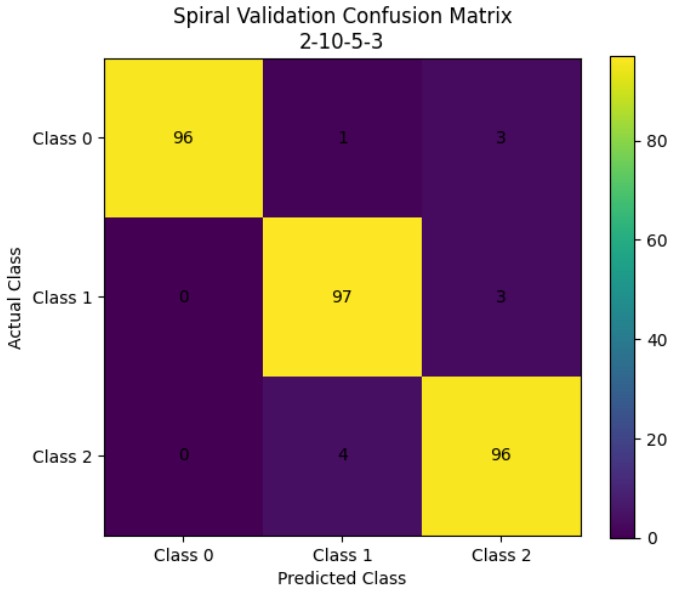
4.4 Average Error Versus Epochs

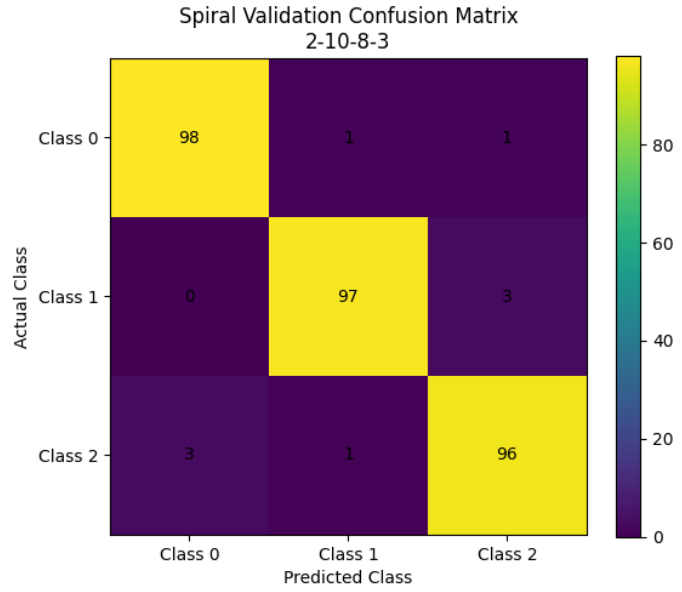


4.5 Validation Performance and Confusion Matrices





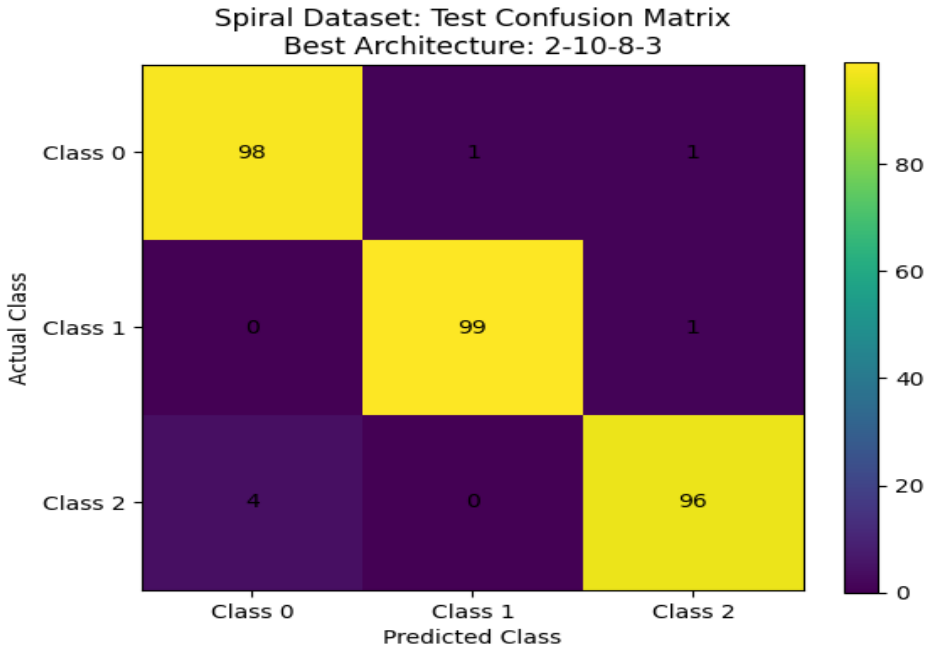




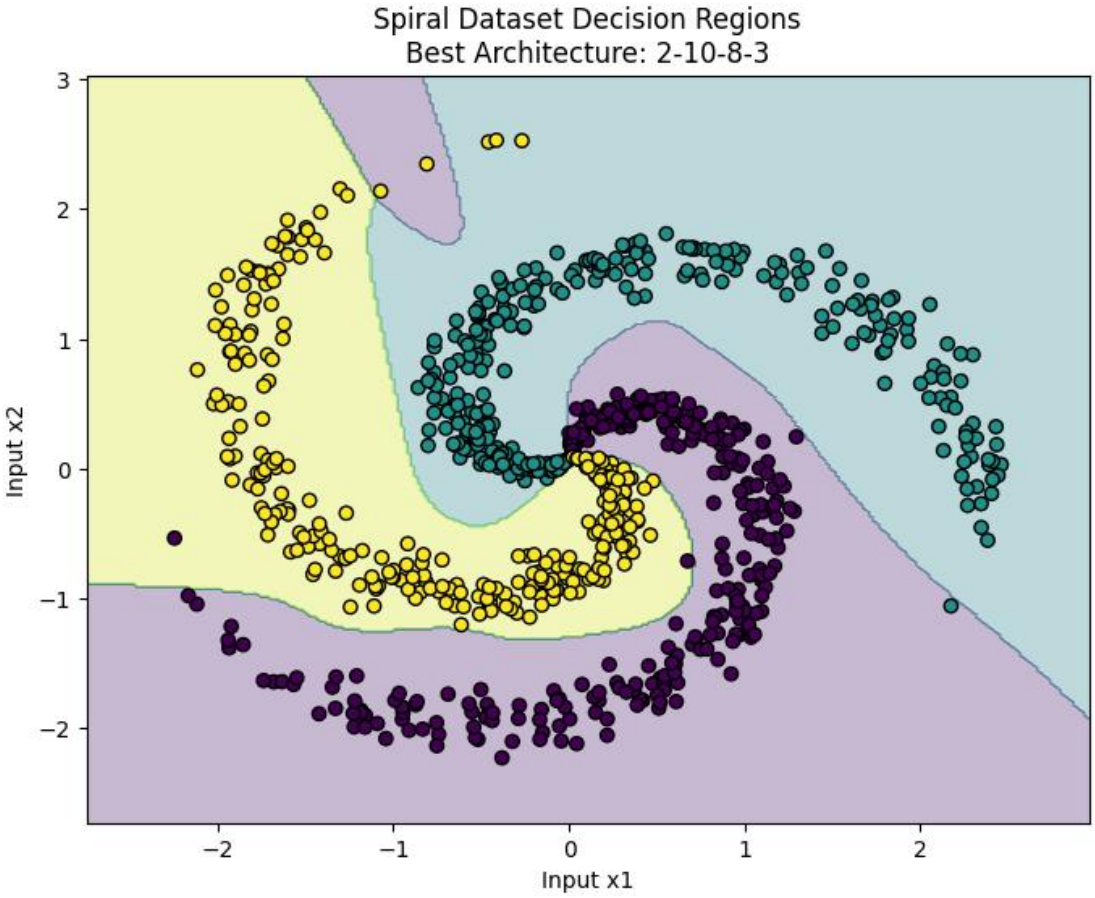
4.6 Final Spiral Test Results

Measure	Result
Best architecture	2-10-8-3
Test accuracy	97.67%
Test samples	300

The test confusion matrix was:



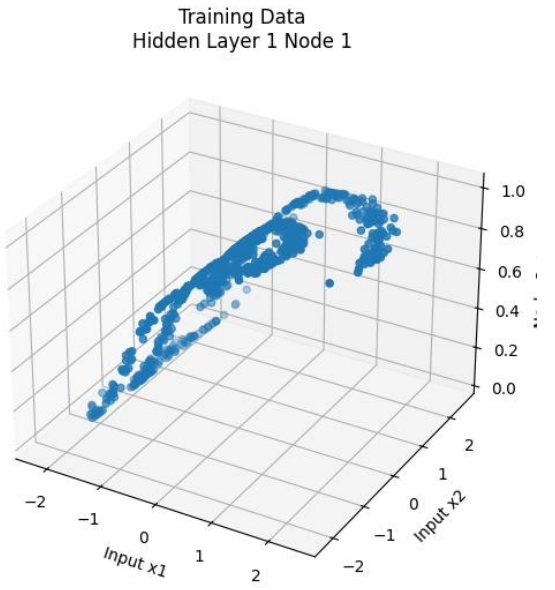
4.7 Spiral Decision Region

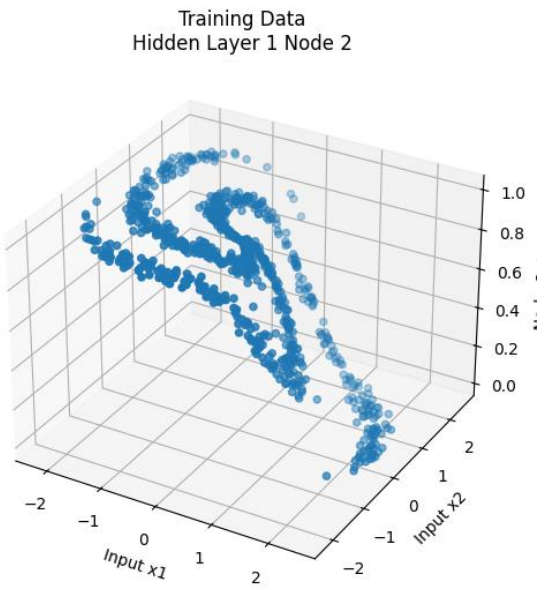


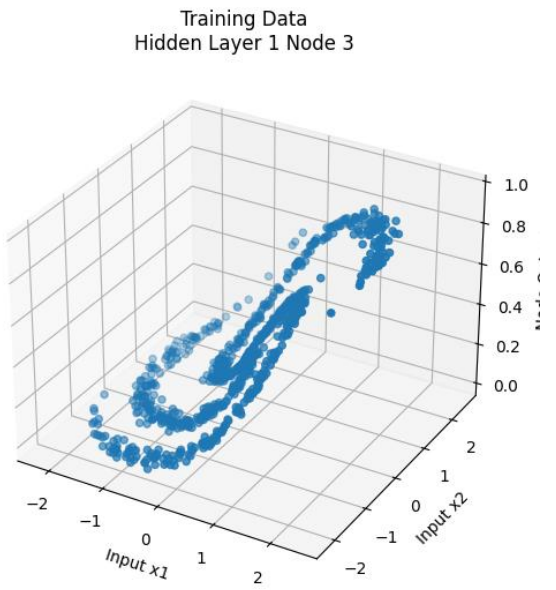
4.8 Spiral Hidden and Output Node Visualizations

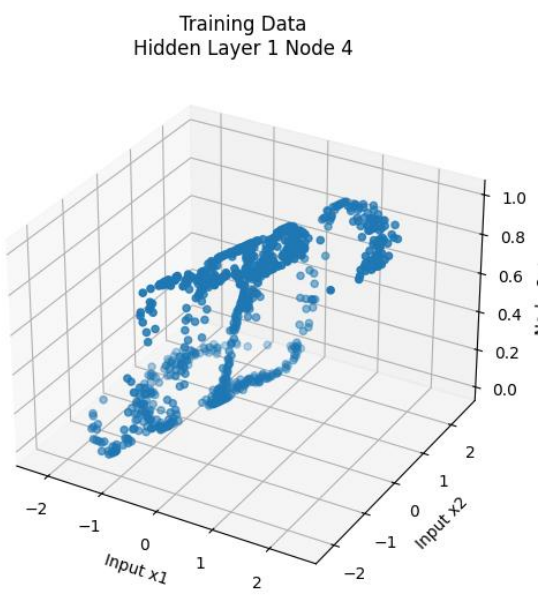
The following figures are the complete hidden-layer and output-node visualizations generated by the final notebook.

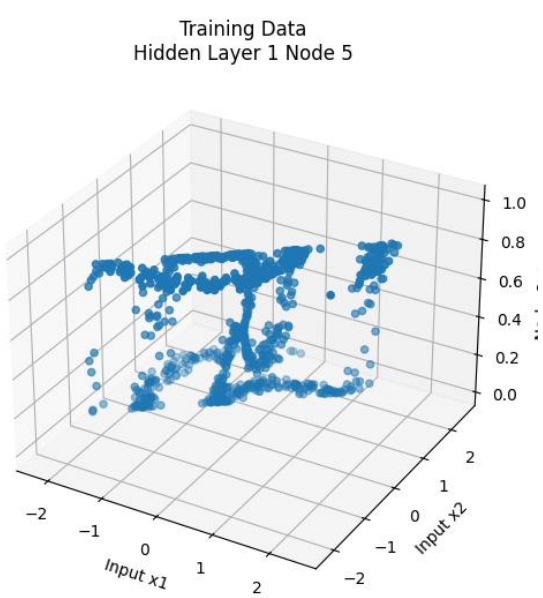
Spiral Training Data Node Visualizations

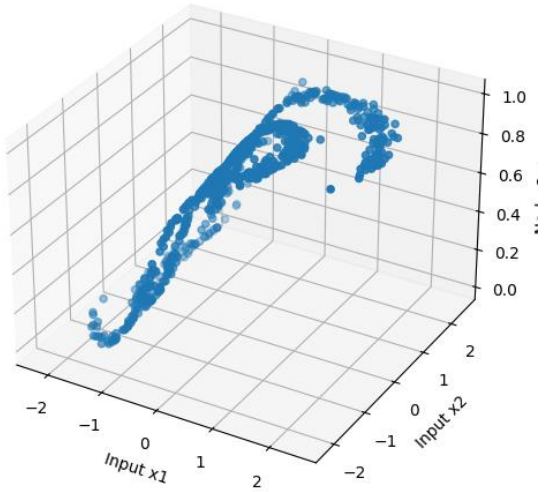




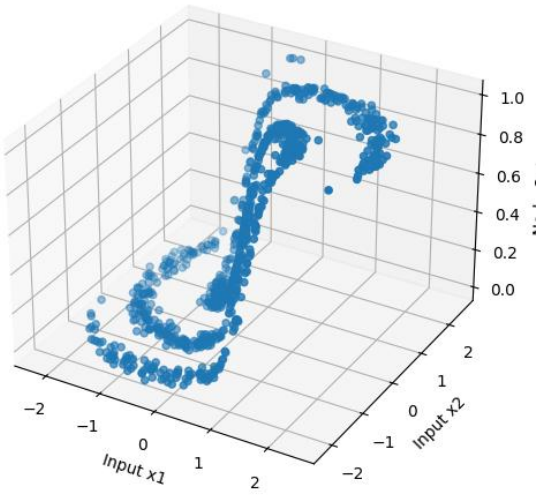




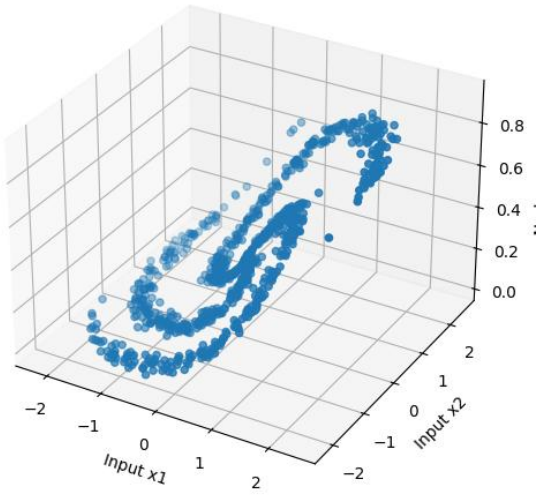




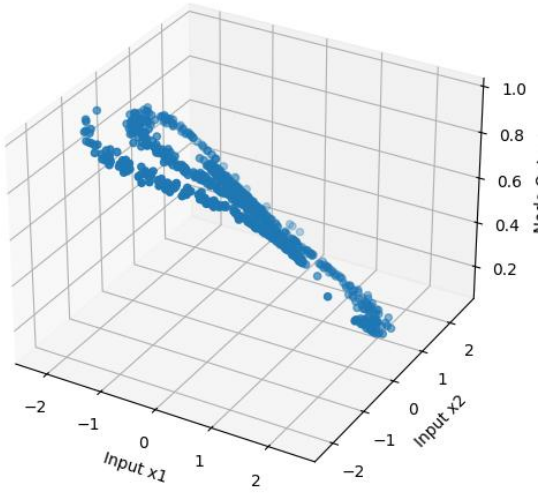
Training Data
Hidden Layer 1 Node 7



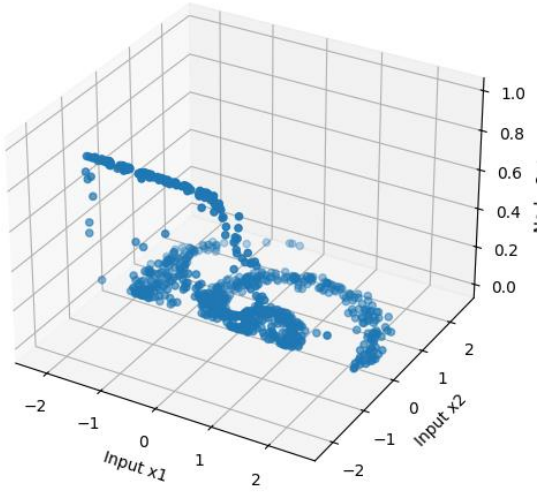
Training Data
Hidden Layer 1 Node 8



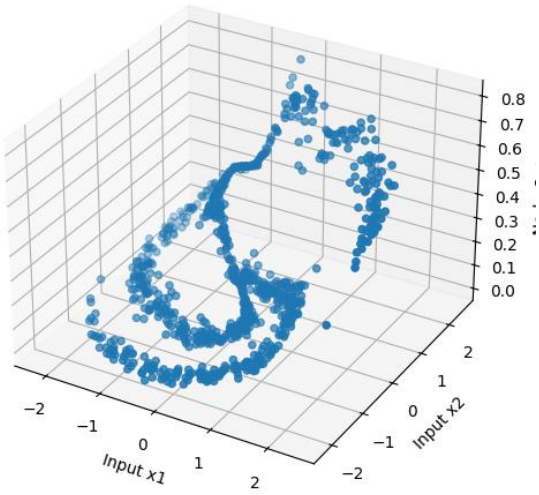
Training Data
Hidden Layer 1 Node 9



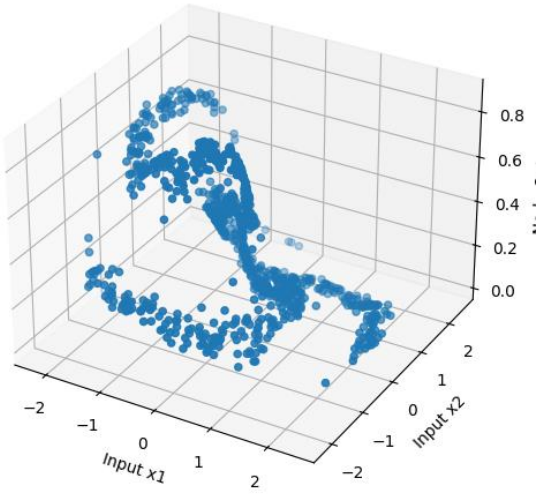
Training Data
Hidden Layer 1 Node 10



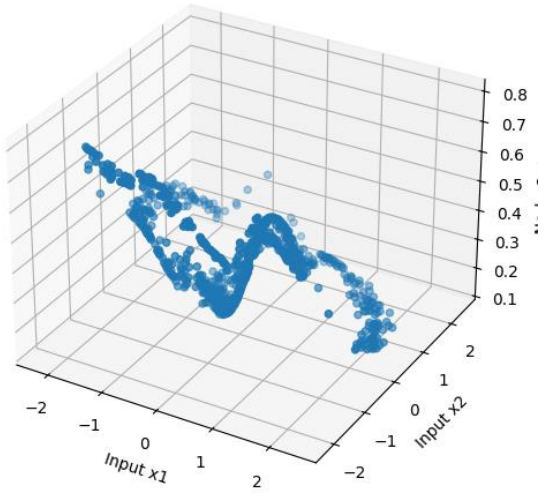
Training Data
Hidden Layer 2 Node 1



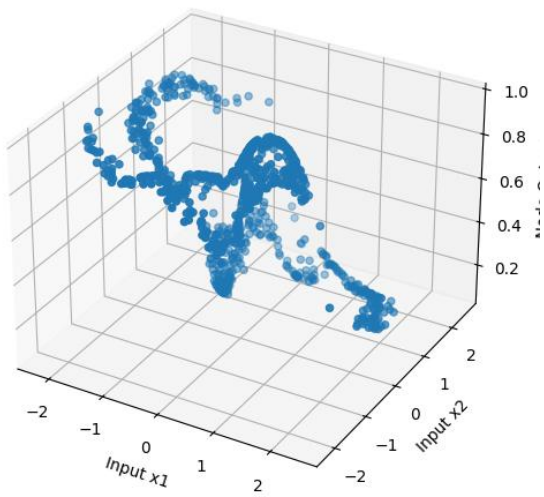
Training Data
Hidden Layer 2 Node 2



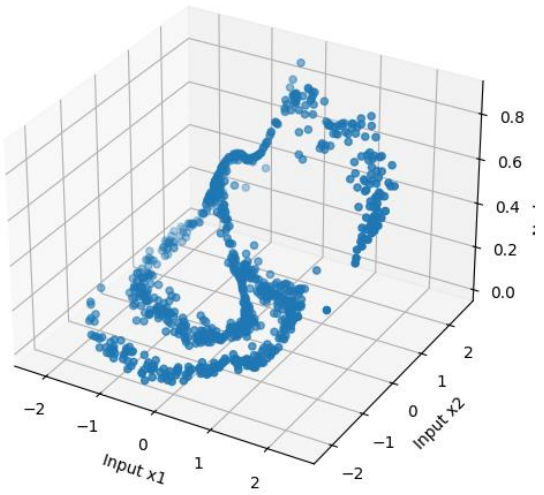
Training Data
Hidden Layer 2 Node 3



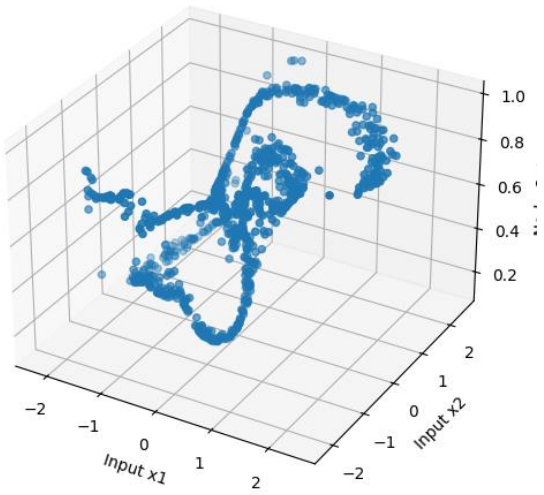
Training Data
Hidden Layer 2 Node 4



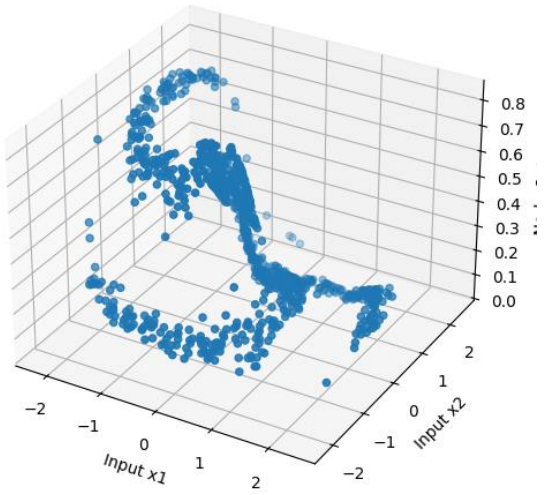
Training Data
Hidden Layer 2 Node 5



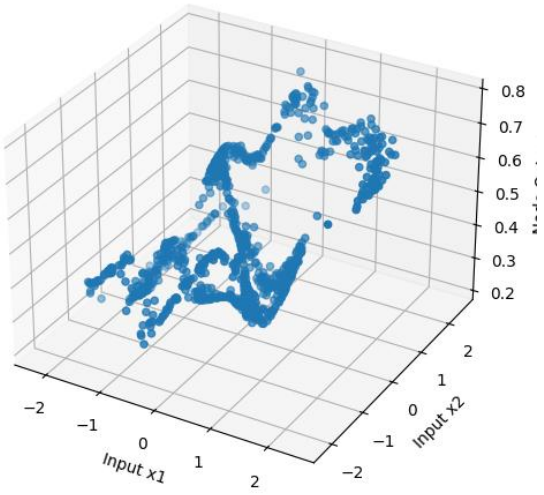
Training Data
Hidden Layer 2 Node 6

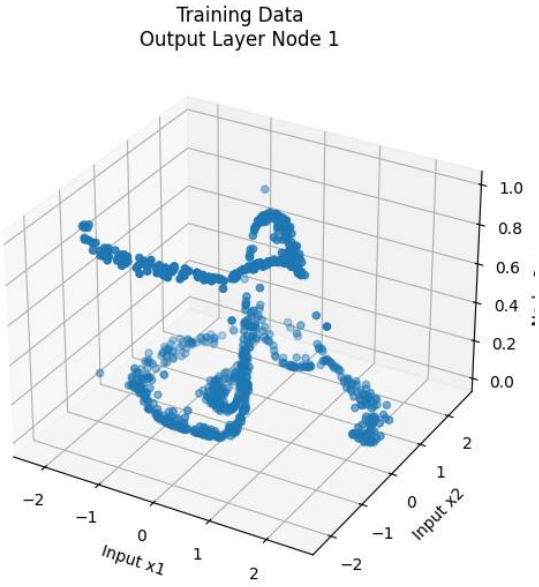


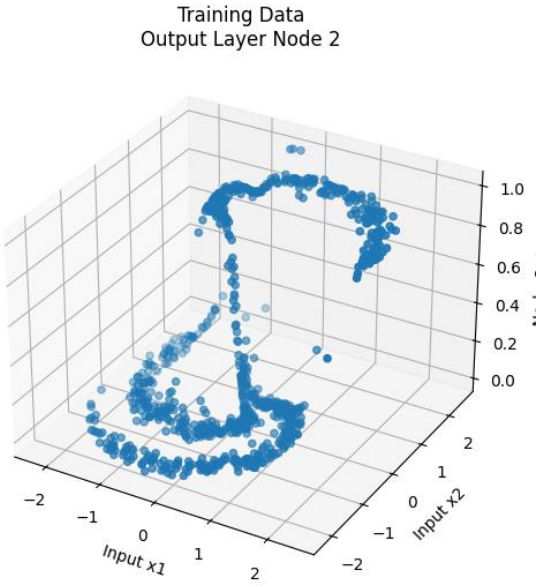
Training Data
Hidden Layer 2 Node 7

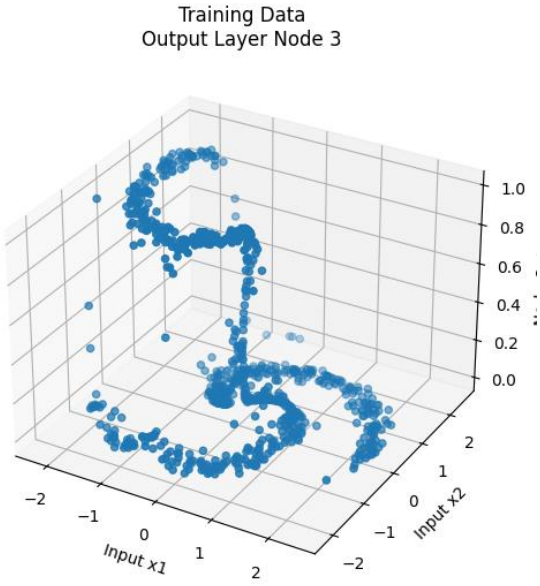


Training Data
Hidden Layer 2 Node 8

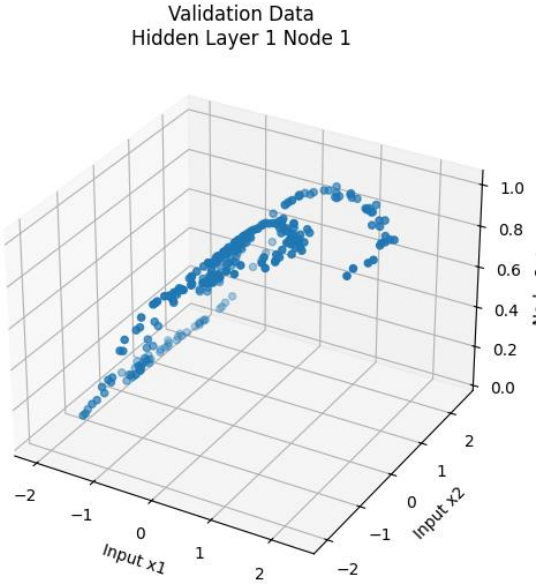


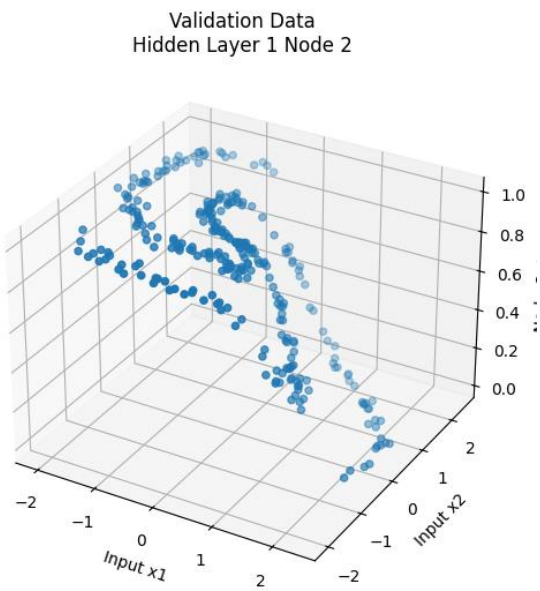




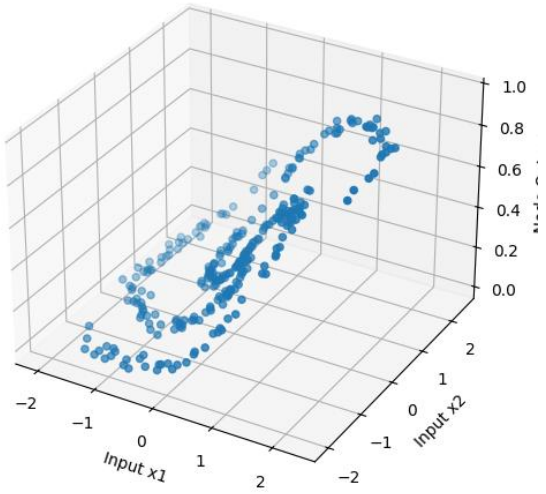


Spiral Validation Data Node Visualizations

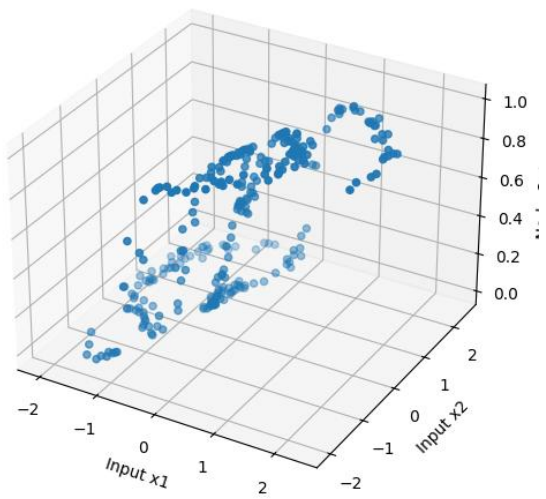




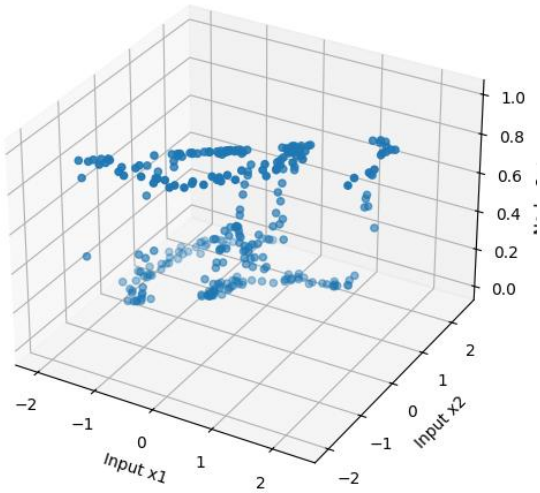
Validation Data
Hidden Layer 1 Node 3



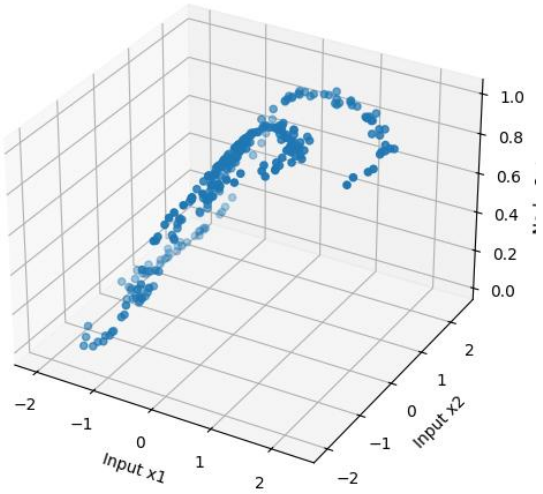
Validation Data
Hidden Layer 1 Node 4



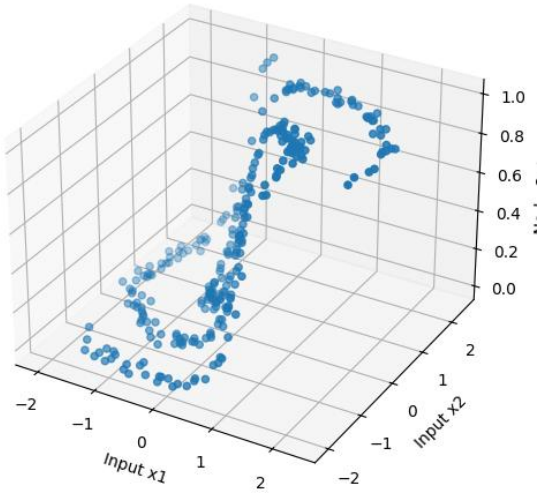
Validation Data
Hidden Layer 1 Node 5



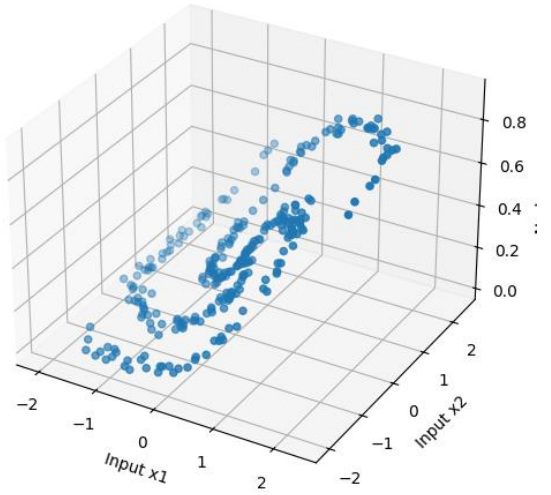
Validation Data
Hidden Layer 1 Node 6



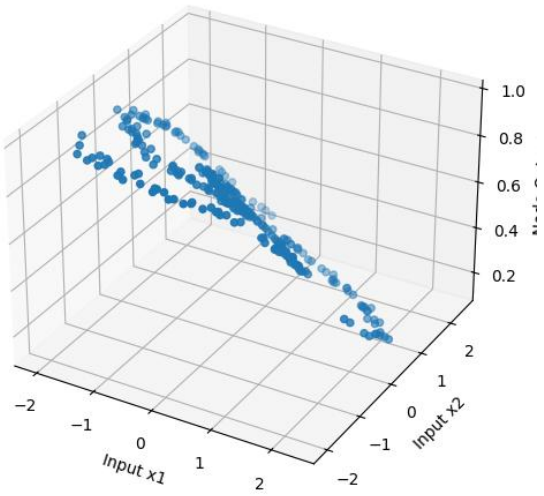
Validation Data
Hidden Layer 1 Node 7



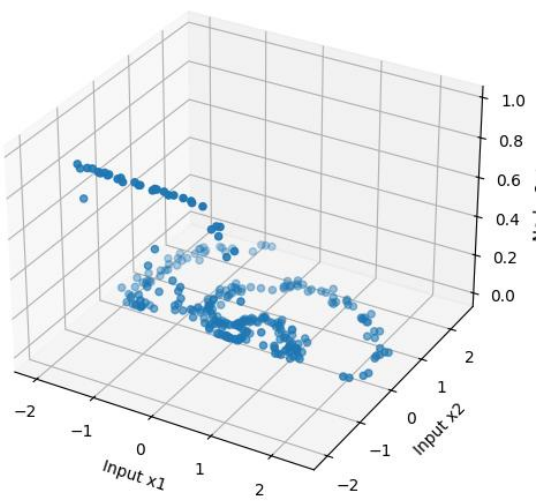
Validation Data
Hidden Layer 1 Node 8



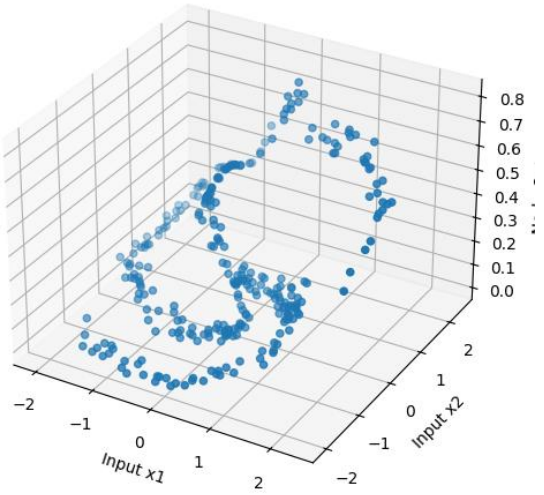
Validation Data
Hidden Layer 1 Node 9



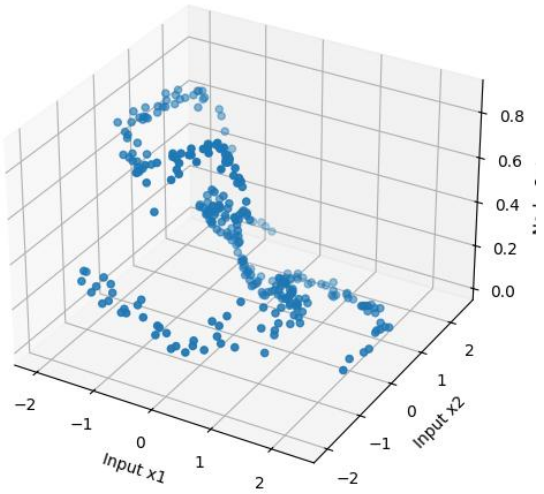
Validation Data
Hidden Layer 1 Node 10



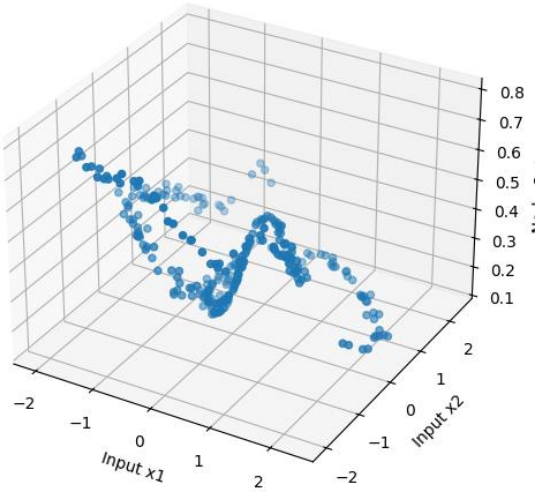
Validation Data
Hidden Layer 2 Node 1



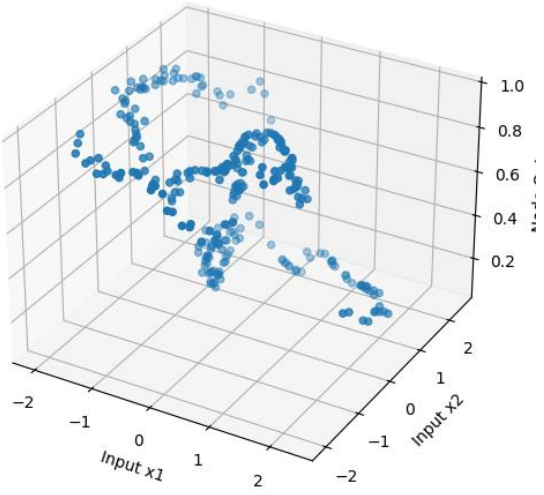
Validation Data
Hidden Layer 2 Node 2



Validation Data
Hidden Layer 2 Node 3

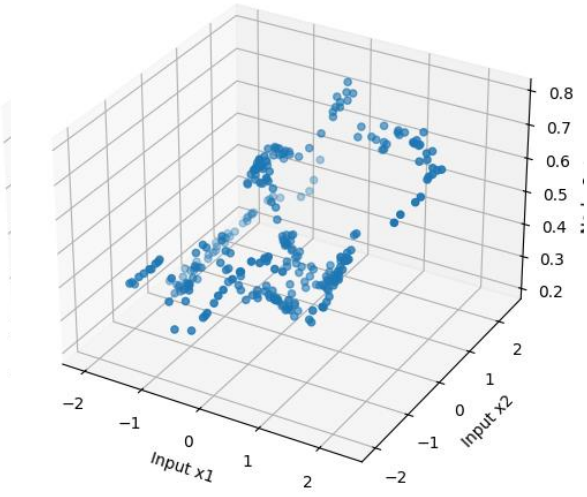


Validation Data
Hidden Layer 2 Node 4

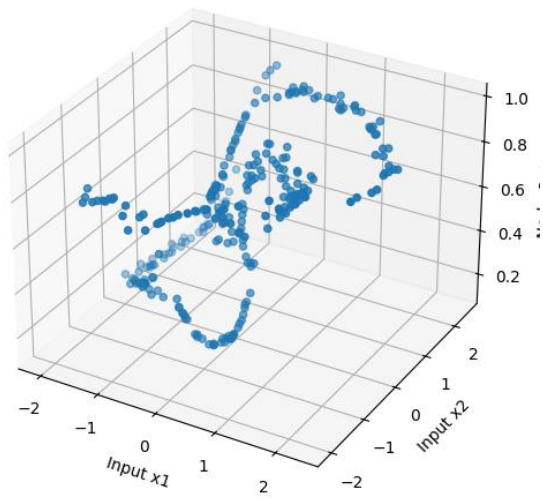


Validation Data

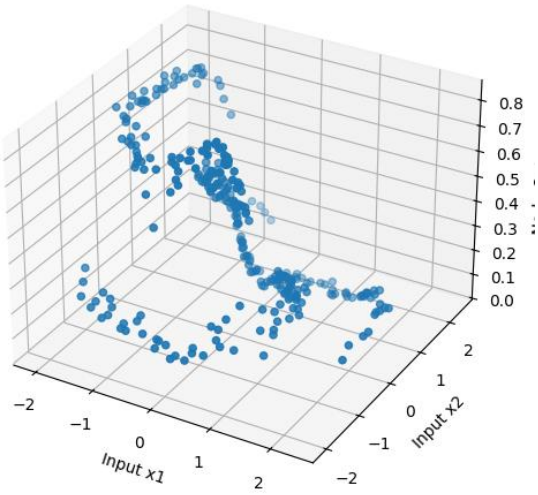
Validation Data
Hidden Layer 2 Node 8



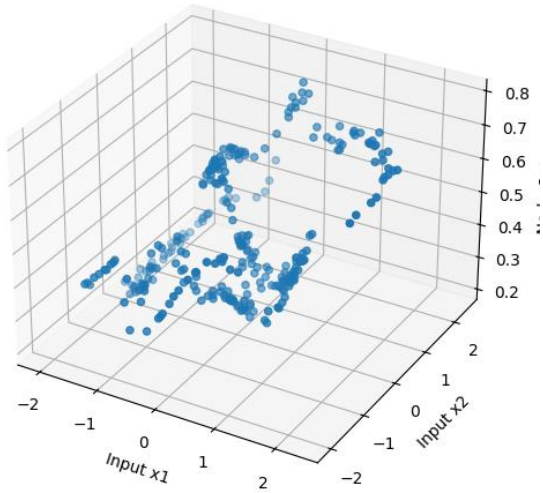
Validation Data
Hidden Layer 2 Node 6



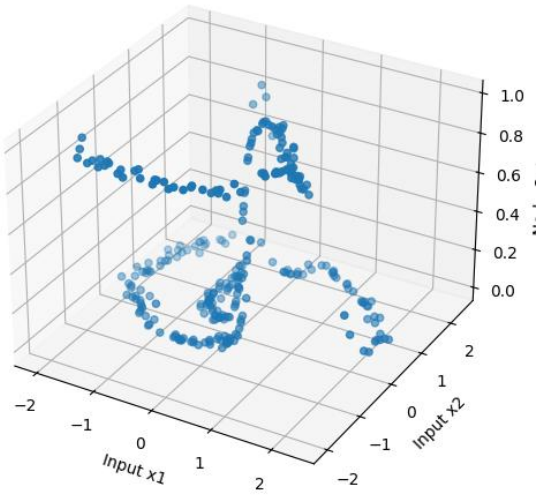
Validation Data
Hidden Layer 2 Node 7



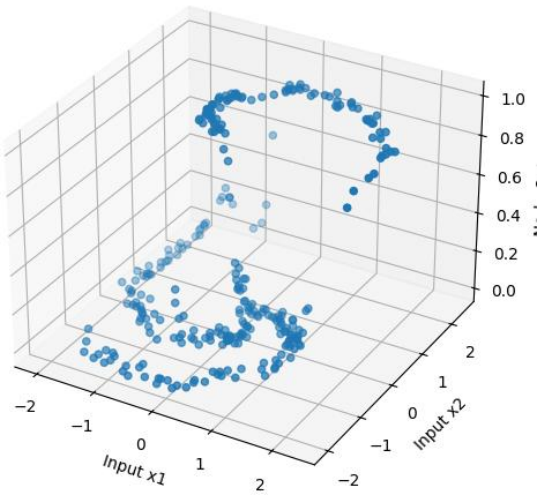
Validation Data
Hidden Layer 2 Node 8

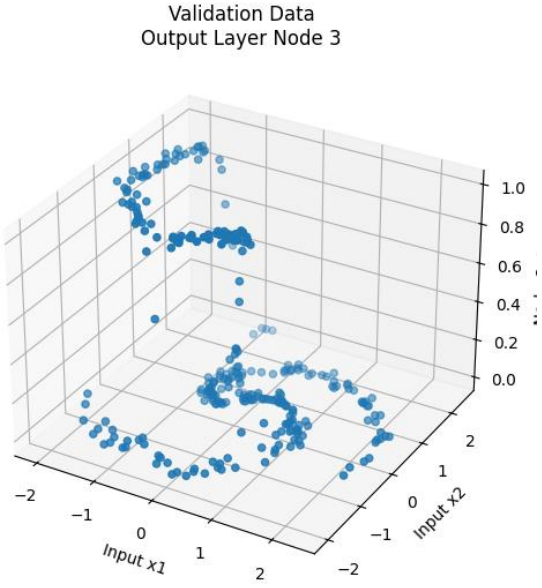


Validation Data
Output Layer Node 1

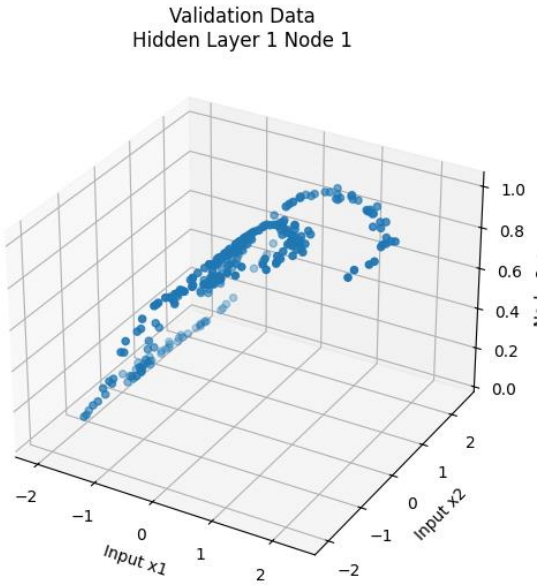


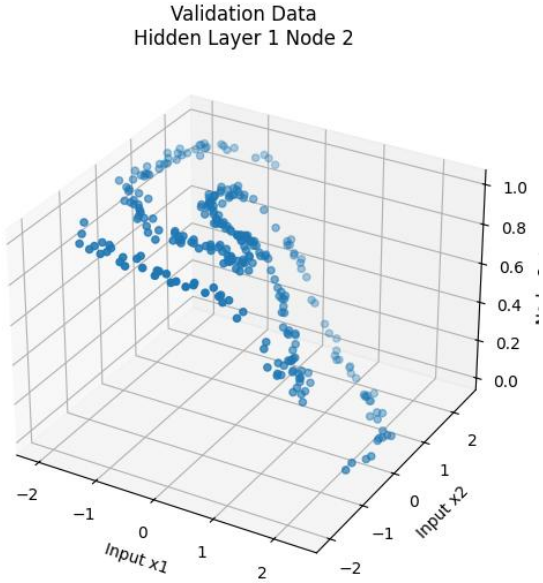
Validation Data
Output Layer Node 2

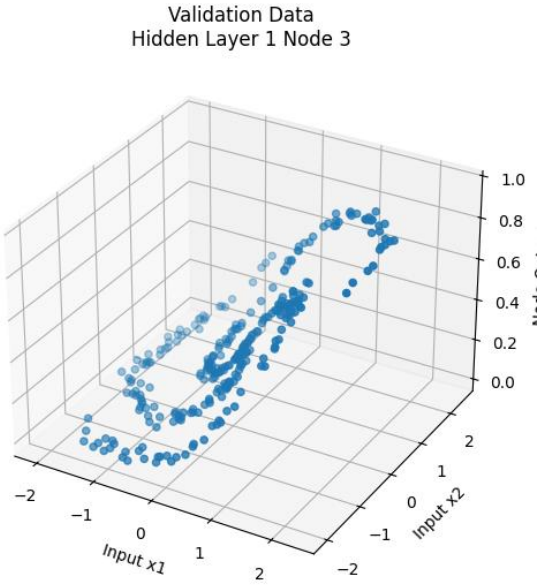


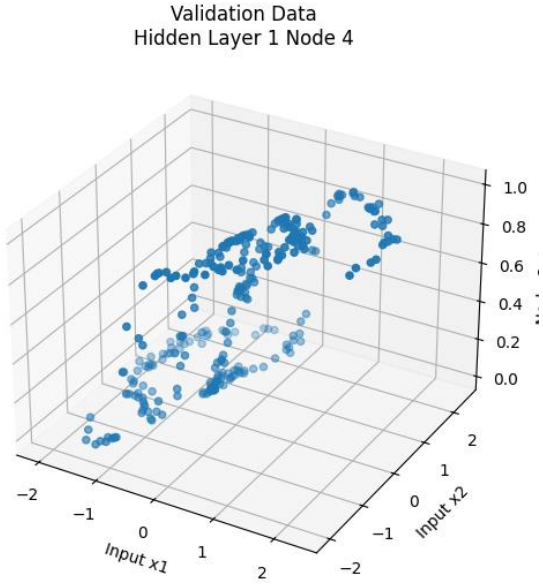


Spiral Test Data Node Visualizations

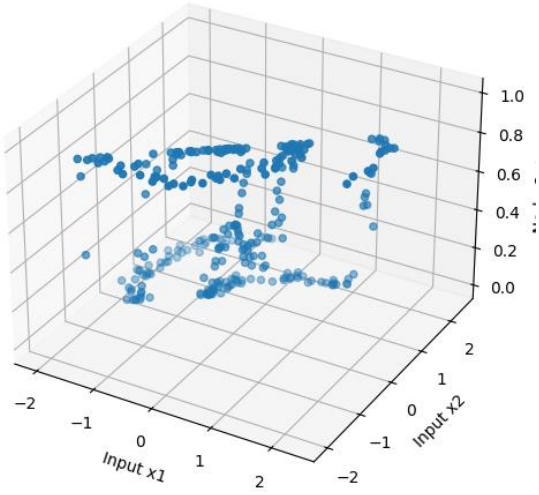




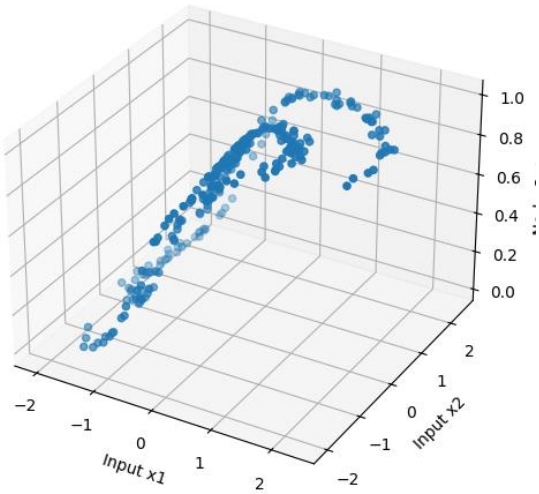




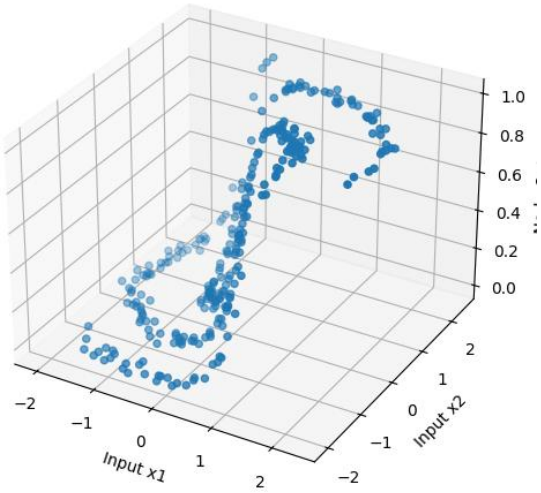
Validation Data
Hidden Layer 1 Node 5



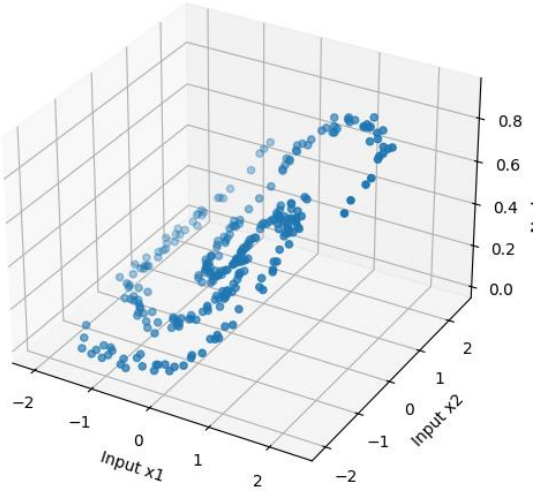
Validation Data
Hidden Layer 1 Node 6



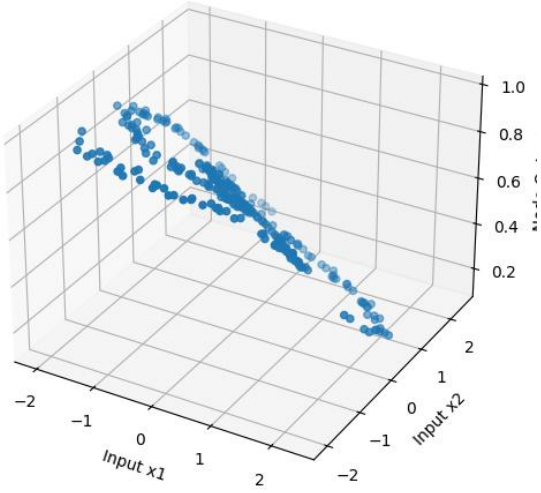
Validation Data
Hidden Layer 1 Node 7



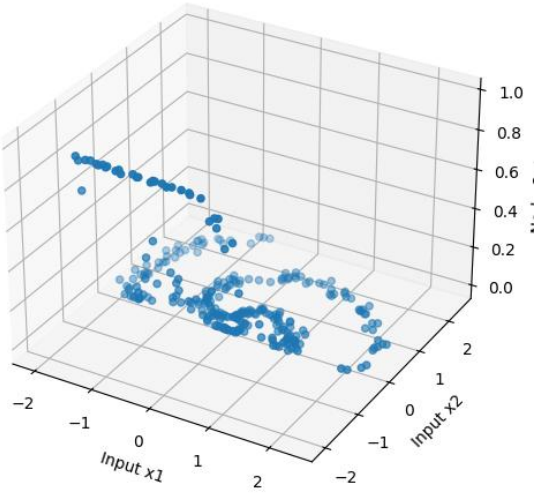
Validation Data
Hidden Layer 1 Node 8



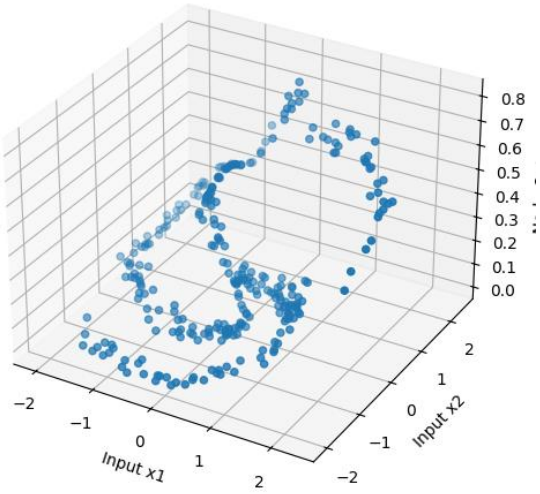
Validation Data
Hidden Layer 1 Node 9



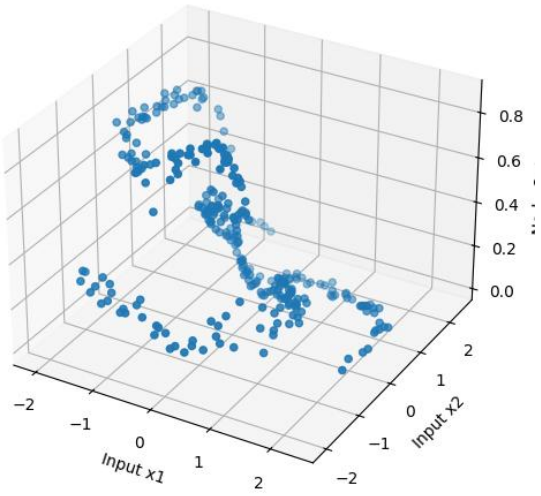
Validation Data
Hidden Layer 1 Node 10



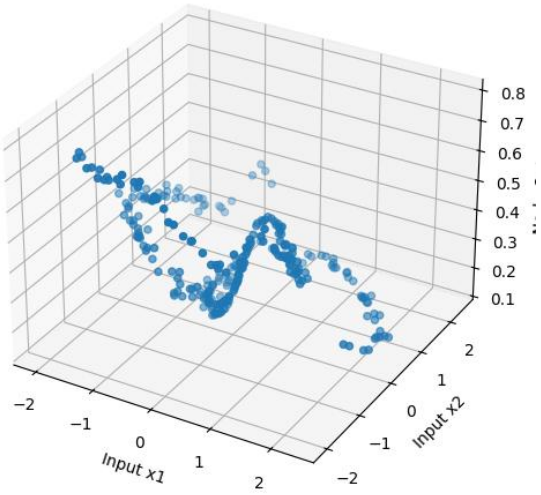
Validation Data
Hidden Layer 2 Node 1



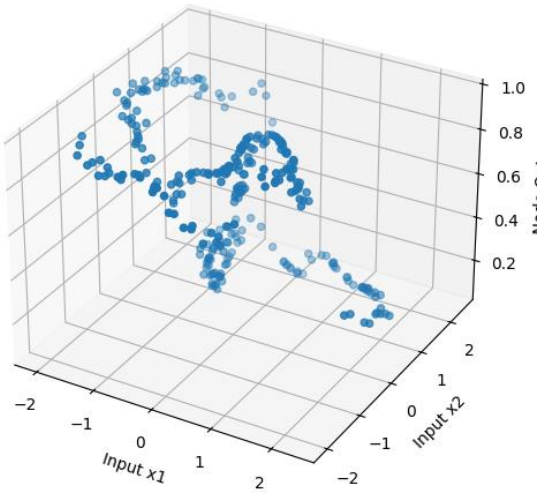
Validation Data
Hidden Layer 2 Node 2



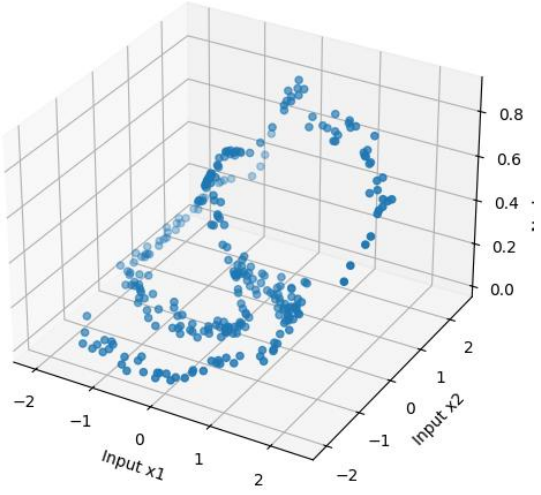
Validation Data
Hidden Layer 2 Node 3



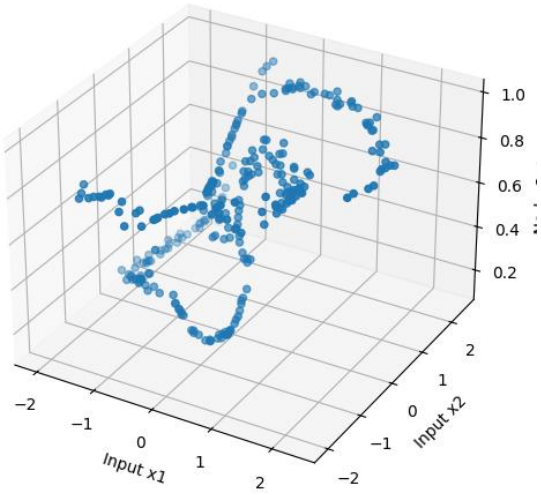
Validation Data
Hidden Layer 2 Node 4



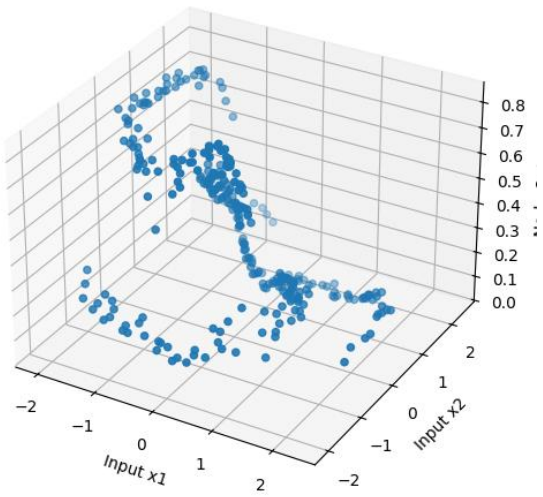
Validation Data
Hidden Layer 2 Node 5



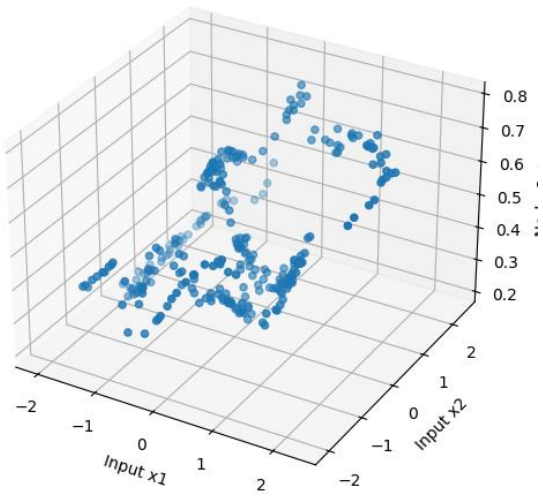
Validation Data
Hidden Layer 2 Node 6



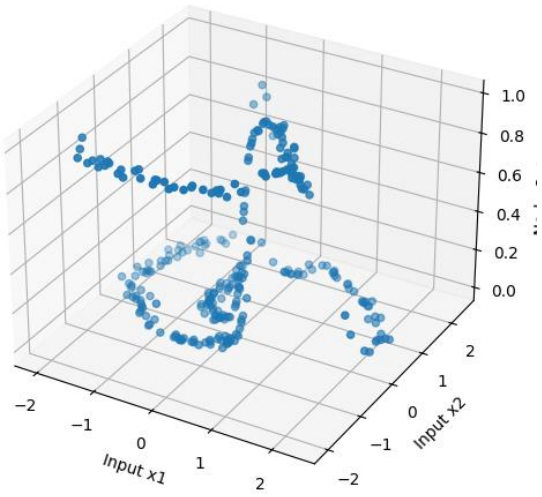
Validation Data
Hidden Layer 2 Node 7



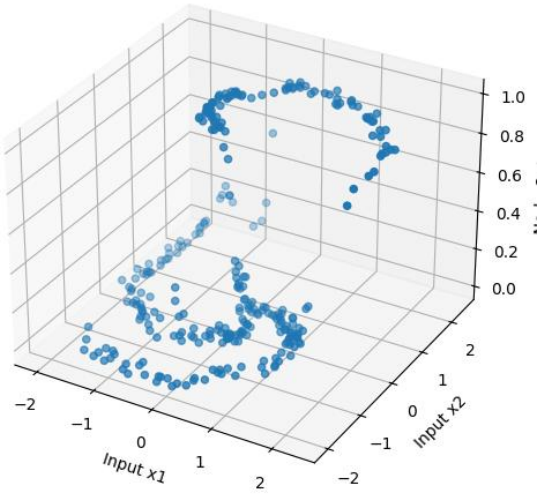
Validation Data
Hidden Layer 2 Node 8



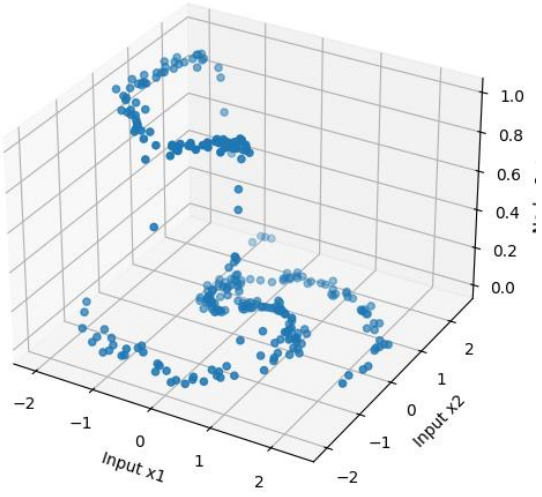
Validation Data
Output Layer Node 1



Validation Data
Output Layer Node 2



Validation Data
Output Layer Node 3



5. Comparison with Single-Neuron Models

Dataset	Single-Neuron Test Accuracy	Best FCNN Test Accuracy	Difference
Gaussian	100.00%	100.00%	0.00 percentage points
Spiral	57.67%	97.67%	40.00 percentage points

For the Gaussian dataset, the single-neuron model and the best FCNN both achieved 100% testing accuracy. This indicates that the Gaussian classes were sufficiently simple for a linear classifier.

For the spiral dataset, the single-neuron model achieved only 57.67% testing accuracy, while the best FCNN achieved 97.67%. The 40 percentage-point improvement demonstrates the importance of hidden layers for learning complex nonlinear decision boundaries.

6. Final Comparison of the Two Datasets

Characteristic	Gaussian Dataset	Spiral Dataset
Dataset size	1500 samples	1500 samples
Input dimensions	2	2
Number of classes	3	3
Network depth used	One hidden layer	Two hidden layers
Best architecture	2-10-3	2-10-8-3
Best validation accuracy	100.00%	97.00%
Best test accuracy	100.00%	97.67%
Single-neuron test accuracy	100.00%	57.67%
Classification difficulty	Lower	Higher

7. Discussion

The Gaussian dataset produced perfect validation and testing performance for the selected FCNN. All tested Gaussian architectures reached 100% validation accuracy, showing that the problem was comparatively easy. The final average error was therefore used to select the 2-10-3 architecture among the tied models.

The spiral dataset was more challenging because its class regions are nonlinear. The deeper two-hidden-layer architecture 2-10-8-3 achieved the strongest validation result of 97.00% and a testing accuracy of 97.67%. The substantial difference between the single-neuron result (57.67%) and the FCNN result (97.67%) shows that multiple hidden layers and nonlinear activations provide much greater representational power.

8. Conclusion

This task demonstrated the complete implementation and evaluation of fully connected neural networks for multi-class classification. The final results showed that FCNNs can achieve excellent performance on both simple and nonlinear datasets when an appropriate architecture is selected. The Gaussian dataset was solved perfectly, while the spiral dataset required a deeper architecture to achieve high accuracy. The experiments also demonstrated that model selection should be based on validation performance, while the test set should be reserved for final evaluation.

Appendix : Recorded Training Outputs

Gaussian Architecture Training Log

==========

Training architecture: 2 -> 3 -> 3
Epoch 20, Average Error: 0.087173
Epoch 40, Average Error: 0.028909
Epoch 60, Average Error: 0.017155
Epoch 80, Average Error: 0.012422
Epoch 100, Average Error: 0.009919
Epoch 120, Average Error: 0.008387
Epoch 140, Average Error: 0.007359
Epoch 160, Average Error: 0.006623
Epoch 180, Average Error: 0.006071
Epoch 200, Average Error: 0.005642

==========

Training architecture: 2 -> 5 -> 3
Epoch 20, Average Error: 0.034587
Epoch 40, Average Error: 0.014044
Epoch 60, Average Error: 0.009138
Epoch 80, Average Error: 0.007015
Epoch 100, Average Error: 0.005838
Epoch 120, Average Error: 0.005091
Epoch 140, Average Error: 0.004575
Epoch 160, Average Error: 0.004196
Epoch 180, Average Error: 0.003906
Epoch 200, Average Error: 0.003676

==========

Training architecture: 2 -> 8 -> 3
Epoch 20, Average Error: 0.028057
Epoch 40, Average Error: 0.012490
Epoch 60, Average Error: 0.008479
Epoch 80, Average Error: 0.006674
Epoch 100, Average Error: 0.005652
Epoch 120, Average Error: 0.004995
Epoch 140, Average Error: 0.004537
Epoch 160, Average Error: 0.004199
Epoch 180, Average Error: 0.003939
Epoch 200, Average Error: 0.003732

==========

Training architecture: 2 -> 10 -> 3
Epoch 20, Average Error: 0.024937
Epoch 40, Average Error: 0.011129
Epoch 60, Average Error: 0.007641
Epoch 80, Average Error: 0.006075
Epoch 100, Average Error: 0.005184

Epoch 120, Average Error: 0.004608
Epoch 140, Average Error: 0.004204
Epoch 160, Average Error: 0.003904
Epoch 180, Average Error: 0.003671
Epoch 200, Average Error: 0.003485

Spiral Architecture Training Log

==========

Architecture: 2-5-3-3
Epoch 50, Average Error: 0.264747
Epoch 100, Average Error: 0.230348
Epoch 150, Average Error: 0.182013
Epoch 200, Average Error: 0.147158
Epoch 250, Average Error: 0.125579
Epoch 300, Average Error: 0.101758
Epoch 350, Average Error: 0.075766
Epoch 400, Average Error: 0.059292
Epoch 450, Average Error: 0.050504
Epoch 500, Average Error: 0.045307

==========

Architecture: 2-8-5-3
Epoch 50, Average Error: 0.264505
Epoch 100, Average Error: 0.225971
Epoch 150, Average Error: 0.163398
Epoch 200, Average Error: 0.098840
Epoch 250, Average Error: 0.063000
Epoch 300, Average Error: 0.047485
Epoch 350, Average Error: 0.039515
Epoch 400, Average Error: 0.034430
Epoch 450, Average Error: 0.030866
Epoch 500, Average Error: 0.028155

==========

Architecture: 2-10-5-3
Epoch 50, Average Error: 0.240543
Epoch 100, Average Error: 0.203172
Epoch 150, Average Error: 0.143213
Epoch 200, Average Error: 0.087429
Epoch 250, Average Error: 0.059277
Epoch 300, Average Error: 0.047264
Epoch 350, Average Error: 0.041095
Epoch 400, Average Error: 0.037273
Epoch 450, Average Error: 0.034604
Epoch 500, Average Error: 0.032461

==========

Architecture: 2-10-8-3
Epoch 50, Average Error: 0.238803
Epoch 100, Average Error: 0.200554
Epoch 150, Average Error: 0.137276
Epoch 200, Average Error: 0.071322
Epoch 250, Average Error: 0.053396
Epoch 300, Average Error: 0.045932
Epoch 350, Average Error: 0.039953
Epoch 400, Average Error: 0.035344
Epoch 450, Average Error: 0.031425
Epoch 500, Average Error: 0.027944